

INSTITUTO TECNOLÓGICO SACABA - ITSa R.M. N°

995/2015

CARRERA DE INFORMÁTICA INDUSTRIAL



PROYECTO DE GRADO

**SISTEMA AUTOMATIZADO DE CONTROL Y DISPENSADO DE
MATERIAL PARA POLARIZADO AUTOMOTRIZ**

POSTULANTE : JHONAR LOZANO PACO

TUTOR: LIC. JOSEFINA ROJAS BALDERAMA

Sacaba - Cochabamba - Bolivia

2026

INSTITUTO TECNOLÓGICO SACABA - ITSa

R.M. Nº 995/2015

CARRERA DE INFORMÁTICA INDUSTRIAL



PROYECTO DE GRADO

**SISTEMA AUTOMATIZADO DE CONTROL Y DISPENSADO DE
MATERIAL PARA POLARIZADO AUTOMOTRIZ**

Modalidad de Titulación presentada como Requisito Parcial para la Obtención del Título
Profesional de Técnico Superior en Informática Industrial

POSTULANTE : JHONAR LOZANO PACO

TUTOR : LIC. JOSEFINA ROJAS BALDERAMA

Sacaba - Cochabamba - Bolivia

2025

DEDICATORIA

Este trabajo es dedicado con amor y cariño:

A mi querida madre, Martha Mery Paco Soliz, por ser un claro ejemplo de sinceridad, sentimiento y amor, por estar siempre disponible cuando más necesito de su sabiduría, gracias por ser como eres y seguir al lado de cada uno de tus hijos.

A mi querido padre, Mario Lozano Paco, por haberme inculcado los valores morales y el don de la perseverancia para alcanzar las metas trazadas.

A mi hermano Brandon Lozano Paco , que con su ejemplo de hermano supo entenderme y apoyarme en cada momento de tristeza y felicidad.

Jhonar Lozano Paco

AGRADECIMIENTOS

A Dios por haberme dado la vida y poner en mi camino a cada una de las personas que siempre me brindaron su apoyo incondicional, en especial a mi hermosa bandida.

Al Instituto Tecnológico Sacaba - ITSa, por haberme permitido formar parte del mismo y concluir un paso importante en el camino de la profesionalización al haber culminado la Carrera de Informática Industrial.

Al LIC. Josefina Rojas Balerrama, por haberme inculcado principios y valores, que me permitieron superar las adversidades que se presentaron a lo largo de mi formación.

Al Sra Olivia Vargas Mendieta, Rector de la institución y al personal de su dependencia; por haber encaminado la presentación de la presente monografía.

A los señores miembros del tribunal interno y externo, por toda la atención y dedicación prestada, por cooperar en que el trabajo alcance el más alto grado de aceptación con la transmisión de sus conocimientos y experiencia profesional.

Al Ing. Ariel Luis Gruich Arratía, tutor del presente trabajo, quien supo encaminar y consolidar el objetivo planteado, sin descuidar la responsabilidad de la función de un tutor.

Jhonar Lozano Paco

INDICE DE CONTENIDO

Contenido

CAPÍTULO I.....	16
PLANTEAMIENTO DEL PROBLEMA.....	16
1.1 DIAGNOSTICO Y JUSTIFICACIÓN	17
1.1.1 Diagnostico	17
1.1.2 Justificación.....	20
1.2 PLANTEAMIENTO Y FORMULACIÓN DEL PROBLEMA TÉCNICO - TECNOLÓGICO	22
1.2.1 Identificación del Problema	23
1.2.2 Formulación del problema.....	27
1.3 OBJETIVOS: GENERAL Y ESPECÍFICOS	29
1.3.1 Objetivo General	29
1.3.2 Objetivos Específicos	29
1.4 ENFOQUE METODOLÓGICO	29
1.4.1 Alcance Temporal	31
CAPÍTULO II.....	33
MARCO TEÓRICO CONCEPTUAL.....	33
2.1 BASES TEÓRICAS	34
2.1.1 Sistemas de Control Automatizado	34

2.1.2 Internet de las Cosas (IoT).....	34
2.2 HERRAMIENTAS DE HARDWARE	34
2.2.1 Microcontrolador ESP32	34
2.2.2 Motor Paso a Paso 28BYJ-48	36
2.2.2.1 Especificaciones Técnicas	36
2.2.3 Driver ULN2003	38
2.2.4 Pantalla LCD 20x4 con I2C	38
2.2.5 Teclado Matricial 4x4	39
2.3 HERRAMIENTAS DE SOFTWARE	40
2.3.1 Arduino IDE.....	40
2.3.2 Visual Studio Code.....	42
2.3.3 PlatformIO	43
2.4 HERRAMIENTA DE GESTIÓN DE BASE DE DATOS.....	44
2.4.1 SQLite	44
2.4.2 DB Browser for SQLite.....	44
2.5 HERRAMIENTAS DE DESARROLLO WEB.....	46
2.5.1 Express.js.....	46
2.5.2 HTML, CSS y JavaScript.....	47
2.6 HERRAMIENTAS DE SIMULACIÓN DE CIRCUITOS	48
2.6.1 Proteus Design Suite	48

2.7 PROTOCOLOS DE COMUNICACIÓN	49
2.7.1 Protocolo HTTP/HTTPS	49
2.7.2 API REST	50
2.7.3 JSON	50
2.7.4 Telegram Bot API	50
2.8 SERVIDOR Y ENTORNO DE DESARROLLO	51
2.8.1 Node.js	51
2.8.2 Express.js (Servidor)	51
2.8.3 Postman	52
2.9 METODOLOGÍA DE DESARROLLO DE SOFTWARE	52
2.9.1 Modelo DRA	52
2.9.2 Fases del Modelo DRA	53
2.10 METODOLOGÍA DE DESARROLLO DE HARDWARE	55
2.10.1 Prototipado Iterativo	89
2.10.2 Fases de Desarrollo de Hardware	89
2.11 HERRAMIENTAS UML (LENGUAJE DE MODELADO UNIFICADO) ...	89
2.11.1 UML	89
2.11.2 Diagrama de Flujo	57
2.11.3 Diagramas de Casos de Uso	57
2.11.4 Diagramas de Clases	58

2.11.5 Diagramas de Secuencia	¡Error! Marcador no definido.
2.12 MODELADO DE BASE DE DATOS	59
2.12.1 Diagrama Entidad-Relación (E-R)	59
2.13 ARQUITECTURA DEL SISTEMA.....	60
2.14 DEFINICIÓN DE TÉRMINOS BÁSICOS	61
CAPÍTULO III.....	64
PROPUESTA DE INNOVACIÓN O	64
SOLUCIÓN DEL PROBLEMA.....	64
3.1 ELABORACIÓN DE MODELADO DE NEGOCIO ACTUAL Y ALTERNATIVO	65
3.1.1 Modelado de Negocio Actual	65
3.1.2 Modelado de Negocio Alternativo	66
3.2 MODELO DRA (DESARROLLO RAPIDO DE APLICACIONES).....	67
3.2.1 FASE 1: MODELADO DE GESTIÓN	67
3.2.2 FASE 2: MODELADO DE DATOS	76
3.2.2.1 Diagrama de clases	76
3.2.2.3 Diccionario de datos.....	77
3.2.3 FASE 3: MODELADO DE PROCESOS	82
3.2.3.1 Diagrama de colaboración	82
3.3 ARQUITECTURA DEL SISTEMA.....	¡Error! Marcador no definido.
3.3.1 Componentes de Hardware	¡Error! Marcador no definido.

3.3.2 Componentes de Software	¡Error! Marcador no definido.
3.3.3 Flujo de Comunicación	¡Error! Marcador no definido.
3.4 CONCLUSIONES DEL CAPÍTULO	97
BIBLIOGRAFÍA	102
Bibliografía	104
Requerimientos No Funcionales	107
Requerimientos Funcionales	107
Conclusión	108

INDICE DE FIGURAS

Figura 1:Árbol de Problemas.....	25
Figura 2: Microcontrolador ESP32.....	36
Figura 3: Motor paso a paso 28BYJ-48	36
Figura 4: Driver ULN2003	38
Figura 5 : Pantalla LCD 20x4 con I2C	39
Figura 6 : Teclado Matricial 4x4	40
Figura 7 : Arduino IDE.....	41
Figura 8 : Visual Studio Code	42
Figura 9 : PlatformIO	43
Figura 10 : SQLite	44
Figura 11 : DB Browser for SQLite	45
Figura 12 : PHP	47
Figura 13 : HTML, CSS y JavaScript	48
Figura 14: Proteus Design Suite.....	49
Figura 15: Modelo DRA.....	54
Figura 16 : Diagrama de Caso de Uso	58
Figura 17 : Diagrama De Clases	59
Figura 18 : Diagrama de Secuencia.....	¡Error! Marcador no definido.

Figura 19 : Diagrama Entidad-Relacion	60
Figura 20 : Modelo de Negocio Actual	65
Figura 21: Modelo de Negocio Alternativo	66
Figura 22 : Diagrama de Casos de usos - Sistemas Dispensadora IoT	70
Figura 23 : Actores del Sistema	74
Figura 24 : Diagrama de Clases-Sistema Dispensadora IoT	76
Figura 25: Diagrama de Entidad-Relacion-Base de Datos	77
Figura 26 : Diagrama de Colaboracion – Validación de código de empleado	82
Figura 27 : Diagrama de colaboración – dispensado de material.....	83
Figura 28: Diagrama de Colaboracion - consulta de reportes	83
Figura 29 : Diagrama de secuencia - Proceso de Dispensado	Error! Marcador no definido.

ÍNDICE DE TABLAS

Tabla 1 : Causas.....	26
Tabla 2 : Efectos.....	27
Tabla 3: Matriz Metodológica.	30
Tabla 4: Cronograma de Actividades Modalidad de Proyecto de Grado.....	31
Tabla 5 : Simbolos utilizados en los diagramas de flujo.....	57
Tabla 6 : Requerimientos funcionales y no Funcionales	67
Tabla 7 : Especificación de caso de uso - Dispensar Material	71
Tabla 8 : Especificación de caso de uso - Iniciar Sesión Dashboard	72
Tabla 9 : Tabla de Actores	75
Tabla 10 : Diccionario de Datos (Empleados)	78
Tabla 11 : Diccionario de Datos (Historial_transacciones)	79
Tabla 12 : Diccionario de Datos (Configuración)	80
Tabla 13: Diccionario de Datos (usuarios)	81

RESUMEN

El presente proyecto desarrolla un sistema automatizado de control y dispensado de laminas de polarizado automotriz, orientado a resolver la problematica de perdida de material por uso no registrado en talleres del sector. Los metodos tradicionales de supervision visual y confianza en el personal generan perdidas estimadas entre el 15% y 25% del inventario anual, afectando la rentabilidad del negocio y deteriorando el clima laboral por la falta de datos objetivos.

La solución integra un microcontrolador ESP32 con conectividad WiFi, un motor paso a paso 28BYJ-48 con driver ULN2003 para el dispensado de precisión, una pantalla LCD 20x4 con módulo I2C y un teclado matricial 4x4 para la interacción del empleado. El sistema se comunica mediante protocolo HTTP con un servidor Node.js que gestiona la logica de negocio, almacena cada transacción en una base de datos SQLite y notifica al administrador en tiempo real a traves de la API de Telegram Bot.

El desarrollo se estructuro bajo el Modelo DRA (Desarrollo Rapido de Aplicaciones), organizado en tres fases: modelado de gestión, modelado de datos y modelado de procesos. Las pruebas funcionales ejecutadas sobre el prototipo arrojaron resultados satisfactorios en los diez casos de prueba planteados, verificando la autenticación por código PIN, el dispensado con precisión de mas o menos 2 centímetros, el registro automático de transacciones, las notificaciones via Telegram y la generación de reportes PDF desde el panel web administrativo.

Palabras clave: dispensado automático, ESP32, IoT, polarizado automotriz, Node.js, SQLite, Telegram Bot, control de inventario.

INTRODUCCIÓN

La gestión de inventarios en pequeñas y medianas empresas representa uno de los desafíos más críticos para la sostenibilidad y rentabilidad operativa. En el sector de servicios automotrices, específicamente en talleres de raybanizado o polarizado de vidrios, el control del material de trabajo constituye un factor determinante que impacta directamente en los márgenes de ganancia y la viabilidad del negocio. La industria del raybanizado en Bolivia ha experimentado un crecimiento sostenido impulsado por la expansión del parque automotor, sin embargo, mantiene sistemas de gestión tradicionales vulnerables a pérdidas significativas.

El presente proyecto surge de la identificación directa de una problemática recurrente en talleres de raybanizado: el uso no registrado del material por parte de empleados, facilitado por la ausencia de sistemas automatizados de control. Las láminas de polarizado, comercializadas en rollos de alto valor económico (1400 Bs), representan entre el 40% y 60% del costo operativo, convirtiéndose paradójicamente en el activo más vulnerable. Los métodos tradicionales de supervisión visual y confianza en el personal han demostrado ser insuficientes, generando pérdidas estimadas entre 15% y 25% del inventario anual.

La solución propuesta consiste en el desarrollo de un sistema automatizado de dispensado y control de láminas de polarizado mediante tecnología IoT (Internet de las Cosas). Este sistema integra un microcontrolador ESP32, motores paso a paso para dispensado preciso, base de datos SQLite para registro de transacciones, notificaciones en tiempo real vía Telegram, trazabilidad completa de cada uso de material y supervisión remota eliminando la dependencia de presencia física del propietario.

El enfoque metodológico del proyecto se fundamenta en el modelo de desarrollo en DRA, apropiado para proyectos que integran hardware y software con requisitos bien definidos. La implementación del sistema no solo aborda la problemática del robo de material, sino que profesionaliza la gestión operativa del sector, promueve transparencia en las relaciones

laborales, y demuestra la viabilidad de adopción de tecnologías modernas en pequeñas empresas bolivianas. Los resultados esperados incluyen eliminación del uso no autorizado de material, reducción de pérdidas económicas, mejora del clima laboral, y establecimiento de un precedente tecnológico replicable en sectores similares.

CAPÍTULO I

PLANTEAMIENTO DEL PROBLEMA

1.1 DIAGNOSTICO Y JUSTIFICACIÓN

1.1.1 Diagnostico

1.1.1.1 Antecedentes Generales

La gestión de materiales en el sector del **raybanizado automotriz** ha evolucionado significativamente en los últimos años, especialmente en talleres de alto rendimiento ubicados en Estados Unidos. Uno de los ejemplos más destacados es **Bluemax Tint**, una empresa con sede en **California (EE. UU.)**, reconocida por su enfoque profesional en la instalación de láminas de polarizado para automóviles y edificaciones. Según el portal especializado, Bluemax Tint se encuentra entre los talleres más innovadores en la aplicación de técnicas de polarizado y control de calidad en América del Norte (Magazine, 2022).

En este tipo de empresas, la **administración del material** suele estar a cargo de un responsable logístico que controla manualmente el uso de los rollos de película polarizada, los registra en hojas de cálculo y coordina la reposición según el consumo semanal. Este método garantiza trazabilidad, pero **incrementa los costos operativos** debido a la necesidad de personal exclusivo para dicha tarea, además de la posibilidad de errores humanos en el registro o conteo de material (Anderson, 2022).

A nivel técnico, las **láminas de polarizado** utilizadas en talleres de nivel internacional tienen un costo elevado y varían según su tipo, espesor y tecnología UV o cerámica, alcanzando precios de **\$80 a \$250 por rollo**, dependiendo de la marca y proveedor. Por ello, el control del inventario resulta un aspecto crítico en la rentabilidad del negocio, dado que una mala gestión puede representar pérdidas considerables de insumos, tiempo y recursos (Hernández Sampieri, 2019).

Este contexto internacional inspiró el desarrollo del presente proyecto, el cual busca **automatizar el proceso de control y dispensado del material de polarizado** mediante un sistema IoT basado en microcontrolador **ESP32** y motores paso a paso de precisión (Systems, ESP32 Technical

Reference Manual, 2021). De esta forma, se pretende replicar el nivel de gestión profesional de talleres como Bluemax Tint, pero eliminando la necesidad de personal adicional

En países latinoamericanos, particularmente en **Bolivia**, la mayoría de los talleres de raybanizado aún operan con métodos manuales de administración y supervisión directa del propietario. Esto genera vulnerabilidad frente a pérdidas de material y falta de trazabilidad en el uso del inventario. La implementación de sistemas automatizados representa, por tanto, una **oportunidad tecnológica viable y rentable**, que permite modernizar el sector y aumentar su competitividad.

Proceso de control y dispensado del material de polarizado mediante un sistema IoT basado en microcontrolador **ESP32** y motores paso a paso de precisión (Systems, ESP32 Technical Reference Manual, 2021). De esta forma, se pretende replicar el nivel de gestión profesional de talleres como Bluemax Tint, pero eliminando la necesidad de personal adicional y optimizando el registro y control de uso de material en tiempo real.

Otro referente internacional es 3M Automotive Window Films, que implementó desde 2018 un sistema de dispensado electrónico controlado por peso y longitud en sus centros de distribución. Este sistema registra automáticamente el material utilizado por cada operario mediante códigos de identificación, generando reportes diarios de consumo que permitieron reducir el desperdicio en un 31% en sus operaciones latinoamericanas (3M Corporation, 2020).

En el ámbito académico, la Universidad Politécnica de Valencia (España) desarrolló en 2021 un prototipo de dispensador inteligente para materiales laminares en entornos industriales, utilizando microcontroladores ARM Cortex y comunicación MQTT para integración con sistemas ERP. El estudio demostró que la automatización del dispensado reduce el tiempo de preparación de materiales en un 42% y elimina prácticamente los errores de medición manual (García y Martínez, 2021).

1.1.1.2 Antecedentes Específicos

En el contexto boliviano, los talleres dedicados al **raybanizado automotriz** presentan una administración tradicional del material de trabajo. La mayoría de estos establecimientos utilizan métodos de control manual y supervisión directa del propietario, basados principalmente en la confianza hacia los empleados y la inspección visual del inventario. Este tipo de gestión suele ser impreciso y vulnerable a pérdidas por uso no controlado del material, desperdicio o errores humanos (Hernández Sampieri, Metodología de la investigación (6ª ed.), 2014).

Un ejemplo recurrente se observa en pequeños talleres locales donde las **láminas de polarizado**, comercializadas en rollos de entre 30 y 50 metros, se cortan manualmente sin registro de uso por empleado ni trazabilidad del consumo diario. Esto genera diferencias entre el material adquirido y el utilizado, provocando pérdidas estimadas del 15 % al 25 % del inventario anual (Anderson, 2022). La ausencia de control automatizado no solo afecta la rentabilidad, sino que también genera desconfianza entre propietarios y trabajadores, deteriorando el ambiente laboral (Gard., 2023).

Esta situación motivó la búsqueda de una alternativa tecnológica aplicable al contexto local. En ese sentido, el presente proyecto propone el **diseño de un sistema automatizado de dispensado y control de láminas de polarizado** basado en tecnología IoT, microcontrolador **ESP32** y motores paso a paso, que permita reducir el margen de error humano, eliminar el uso no autorizado de material y optimizar la administración del inventario (Systems., 2021).

A nivel local, la ciudad de Sacaba cuenta con aproximadamente 12 talleres dedicados al raybanizado automotriz según el Registro de Actividades Económicas del Municipio de Sacaba (2024). Estos establecimientos comparten características comunes: operación familiar, personal de entre 2 y 8 empleados, y ausencia total de sistemas automatizados de control de materiales. Las entrevistas realizadas durante el diagnóstico del presente proyecto revelaron que el 83% de los propietarios identifican las pérdidas de material como su principal problema operativo, estimando pérdidas de entre 200 y 600 bolivianos mensuales por este concepto.

En el contexto regional de Cochabamba, el Instituto Tecnológico Sacaba (ITSa) ha impulsado en los últimos ciclos académicos el desarrollo de proyectos de innovación tecnológica orientados a resolver problemáticas del sector productivo local. Proyectos previos de la carrera de Informática Industrial han abordado sistemas de monitoreo con ESP32 y automatización con Arduino, constituyendo una base de conocimiento institucional sobre la cual se fundamenta el presente desarrollo (ITSa, 2023).

1.1.2 Justificación

1.1.2.1 Justificación Teórica

El presente proyecto desarrolla un Sistema Automatizado de Control y Dispensado para láminas de polarizado mediante tecnología IoT (Internet de las Cosas) , utilizando específicamente un microcontrolador ESP32 y motores de precisión . Actualmente, los talleres carecen de control automatizado, lo que resulta en desperdicio de material y pérdidas económicas (estimadas entre 15% y 25%) por "uso no registrado del material". El sistema resuelve esto al controlar automáticamente el acceso y dispensar cantidades exactas, registrando cada transacción con identificación de usuario en una base de datos SQLite y notificando en tiempo real vía Telegram al propietario. Esto transforma la gestión, brindando auditorías precisas y reduciendo pérdidas de manera inmediata.

1.1.2.2 Justificación Social

El proyecto posee una **justificación social sólida** dado que aborda la problemática del deterioro del clima laboral y la desconfianza mutua que surge en el sector de talleres de raybanizado a causa del descontrol de materiales. La falta de transparencia en el manejo del *stock* afecta directamente la **calidad de vida y la integridad profesional** de los empleados honestos, al exponerlos a sospechas infundadas por pérdidas no identificadas.

La implementación del sistema automatizado representa una **alternativa que fortalece la equidad** dentro del taller al establecer un registro objetivo, inmutable y verificable de cada transacción. Este mecanismo elimina la ambigüedad y permite a los trabajadores demostrar su

probidad mediante datos concretos, protegiendo su buen nombre y **elevando su dignidad laboral** . Este impacto social se traduce en:

- **Mejora del Clima Laboral:** Se elimina la fuente de desconfianza, fomentando un ambiente de respeto que reduce el estrés y el conflicto interno.
- **Estabilidad en el Empleo:** Al aumentar la satisfacción y el sentido de justicia, se reduce la rotación de personal, lo cual beneficia la estabilidad económica de los hogares de los empleados.

1.1.2.3 Justificación Técnica

En el ámbito técnico, este proyecto se fundamenta en la necesidad de desarrollar soluciones de **automatización e IoT** modernas y efectivas que optimicen el control de inventario en talleres automotrices. El sistema propuesto integra componentes de **control de movimiento de precisión** y **comunicaciones en tiempo real** , permitiendo una gestión de recursos materiales que supera ampliamente a los métodos tradicionales de corte y registro manual.

A diferencia del manejo convencional, donde la medición y el corte dependen de la habilidad del operador y el registro está sujeto a errores de transcripción, nuestra propuesta aplica:

1. **Control de Movimiento Automatizado:** Se emplean **motor paso a paso 28BYJ-48**, lo que garantiza una **precisión de dispensado de $\pm 2\text{mm}$** , eliminando el desperdicio generado por la inexactitud humana.
2. **Arquitectura IoT Integral:** La solución se basa en el microcontrolador **ESP32** —seleccionado por su bajo costo, eficiencia energética y conectividad **Wi-Fi** integrada—que actúa como el corazón del sistema, integrando la lógica de control del hardware con las plataformas de gestión de datos.

La arquitectura tecnológica del sistema está diseñada para operar de forma confiable mediante la interconexión de una base de datos **SQLite** con un entorno **Node.js** (Developers, 2024), lo cual asegura un registro histórico inalterable de cada transacción. Esta capacidad de **análisis de datos inteligente** y la posibilidad de monitoreo remoto a través de la **API de Telegram Bot** aumentan

considerablemente la **auditoría** del inventario, garantizando una **confiabilidad, escalabilidad y mantenibilidad** superior a largo plazo.

1.1.2.4 Justificación Metodológica

El desarrollo del **Sistema Automatizado de Control y Dispensado** requiere un enfoque metodológico que garantice la integración eficiente del prototipo físico (*hardware*) con las soluciones de gestión de datos (*software*). Para ello, se justifica la adopción del **Modelo de Desarrollo en DRA** , un enfoque de desarrollo de sistemas que permite transformar los requerimientos de control en una solución práctica y verificable en etapas secuenciales.

A diferencia de las metodologías iterativas, la DRA obliga a **completar y validar cada fase** (Análisis, Diseño Físico y Lógico, Implementación, Pruebas) con entregables tangibles, lo cual es fundamental para proyectos con componentes electromecánicos, minimizando los riesgos de **fallas de integración *hardware-software*** en las etapas finales.

Este enfoque se complementa con la técnica de diseño **Top-Down** , que permite abordar la complejidad del proyecto comenzando desde la **visión global del sistema de control y auditoría** , para luego desglosarlo en **módulos específicos y detallados** .

Con esta estrategia, inicialmente se define la arquitectura del sistema de dispensado, considerando la interacción de los principales componentes, como: el **microcontrolador ESP32** , los **motor paso a paso 28BYJ-48** , la **base de datos SQLite** y el subsistema de **notificaciones por Telegram** . Posteriormente, se procede al desarrollo de módulos más pequeños, tales como:

- El diseño de la **interfaz web de monitoreo** y gestión de usuarios.
- El *firmware* para la **autenticación por código PIN** y el control preciso de los motores.
- El subsistema de **comunicación y sincronización de datos** con el servidor.

En síntesis, la justificación metodológica se fundamenta en la necesidad de un enfoque sistemático y estructurado (DRA) que asegure la correcta integración técnica del prototipo,

combinado con una estrategia de diseño modular (Top-Down) que facilite la gestión ordenada y verificable de cada subsistema.

1.2 PLANTEAMIENTO Y FORMULACIÓN DEL PROBLEMA TÉCNICO - TECNOLÓGICO

1.2.1 Identificación del Problema

Se observa que, en los diferentes talleres de raybanizado de Cochabamba (incluyendo Sacaba) y en establecimientos similares a nivel nacional, la gestión y el control del material de trabajo principal (láminas de polarizado) se lleva a cabo mediante métodos completamente manuales y visuales . Este sistema convencional es inherentemente propenso a errores humanos y altamente vulnerable a la manipulación deshonesta , lo que resulta en dos problemas críticos y directamente relacionados: en primer lugar, se generan pérdidas económicas significativas por concepto de "uso no registrado del material" y desperdicio no controlado (estimadas entre el 15% y 25% anual). En segundo lugar, la administración diaria de este material se convierte en una pérdida de tiempo total y perjudicial para los dueños de las sucursales, ya que la verificación física constante impide la disponibilidad inmediata y verificable del *stock* en tiempo real. Esta falta de conocimiento oportuno y preciso sobre el inventario disponible es la causa principal de la ruptura de la cadena de suministro interna , lo que incrementa el riesgo de interrupción del servicio al cliente por falta de material. Este fenómeno resalta la necesidad de implementar un sistema tecnológico que, mediante la automatización y el registro digital, minimice el desperdicio y garantice la transparencia y disponibilidad continua del recurso.

1.2.1.1 Identificación de la Situación Problemática

La vulnerabilidad de los recursos materiales se origina en la **insuficiencia de un sistema de control de acceso y dispensado automatizado** . Al no incorporar mecanismos de validación de identidad y registro electrónico antes de la dispensación, la capacidad de proteger el inventario resulta limitada. Como consecuencia, se facilita el **uso no registrado del material** sistemático de láminas de polarizado, afectando directamente el margen de ganancia operativa.

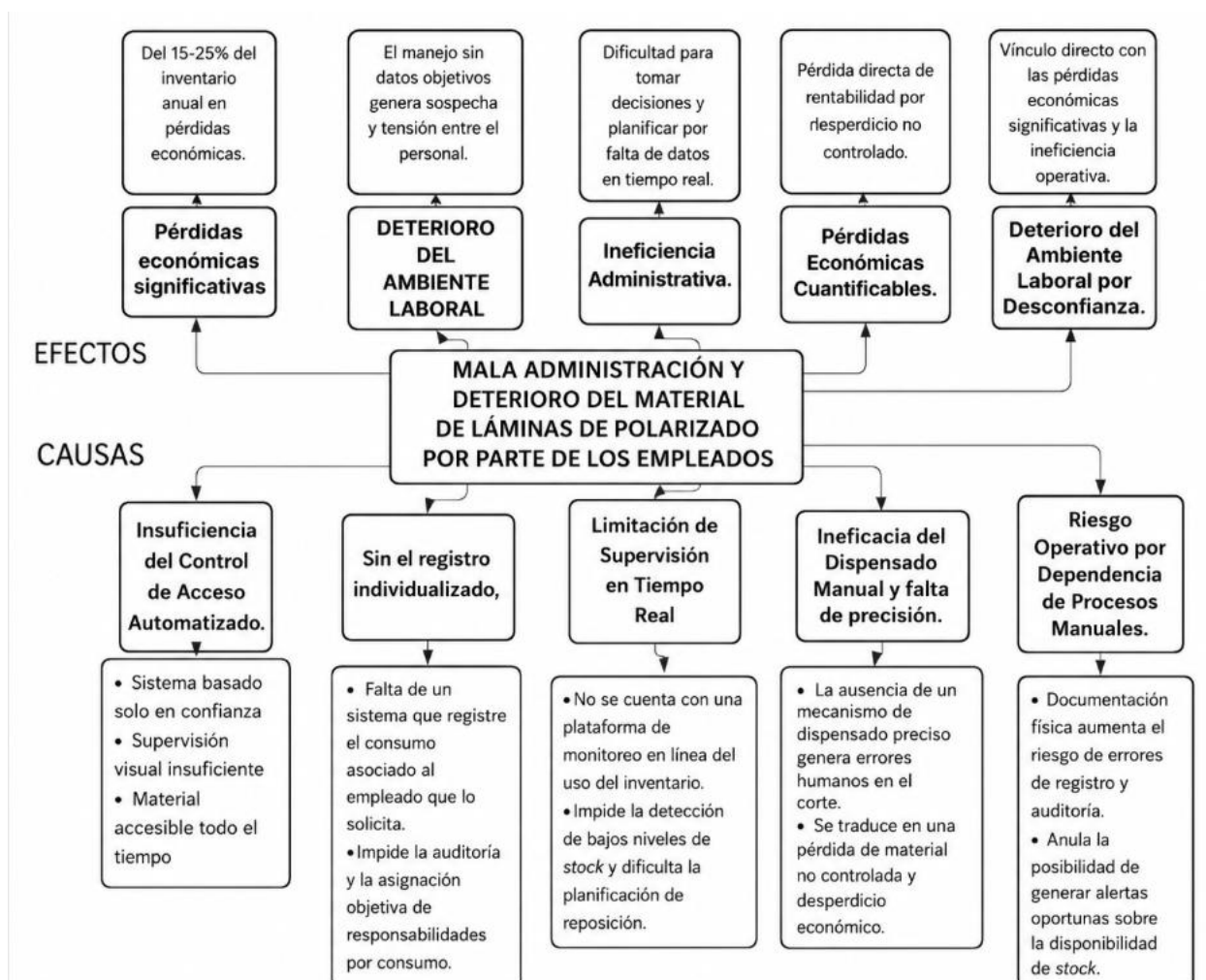
Los procesos de auditoría interna resultan ineficaces debido a la **carencia de un registro de consumo individualizado y asociado al usuario** . Al no diferenciar las transacciones por identidad, la capacidad de establecer la **trazabilidad objetiva** del material utilizado por cada empleado se dificulta. Esto impide la identificación precisa y justa de los responsables de las sirenas o faltantes de inventario.

La eficiencia operativa se ve obstaculizada por la **ausencia de mecanismos de supervisión remota y en tiempo real** del *stock* y su uso. Al no contar con un monitoreo en línea del inventario, se generan **ventanas de oportunidad** para el uso no autorizado o el desperdicio. Más aún, la imposibilidad de recibir alertas inmediatas sobre bajos niveles de *stock* **restringe la planificación de reposición** , manteniendo vulnerables las operaciones ante la potencial **interrupción del servicio al cliente** .

La rentabilidad económica se ve comprometida por la **ineficacia en el control de dispensado y la dependencia de métodos manuales** . Al no incorporar tecnología de dispensado automatizado con precisión milimétrica, se obtiene un **margen de pérdida de inventario** que aumenta anualmente a un estimado de **15% a 25%** debido a la falta de precisión. Este desperdicio es inherente al error humano y no es detectable ni mitigable bajo el sistema actual.

La integridad del capital humano se ve afectada por la **dependencia de la supervisión directa y la falta de datos objetivos de consumo** . Esta vulnerabilidad social genera un **deterioro del clima laboral** caracterizado por la desconfianza y la sospecha infundada hacia el personal. Como resultado, la relación entre el propietario y el personal se tensa, lo que incrementa la **rotación de empleados honestos** y afecta negativamente la estabilidad y productividad del taller.

Figura 1:Árbol de Problemas.



Fuente: Elaboración Propia, 2025.

1.2.1.1 Causas

Tabla 1 : Causas

N°	CAUSA	DESCRIPCIÓN
1	Insuficiencia del Control de Acceso Automatizado	• Sistema basado solo en confianza • Supervisión visual insuficiente • Material accesible todo el tiempo sin restricciones • Facilita el uso no autorizado de material y el uso no autorizado
2	Sin el Registro Individualizado	• Falta de un sistema que registre el consumo asociado al empleado • Impide la auditoría y asignación objetiva de responsabilidades • No hay trazabilidad de quién solicita qué cantidad • Imposibilita detectar patrones de consumo anormales
3	Limitación de Supervisión en Tiempo Real	• No existe plataforma de monitoreo en línea del inventario • Impide la detección de bajos niveles de stock • Dificulta la planificación de reposición • Supervisión reactiva en lugar de preventiva
4	Ineficacia del Dispensado Manual y Falta de Precisión	• Ausencia de mecanismo de dispensado preciso • Genera errores humanos en el corte ($\pm 10-20$ cm)

		• Pérdida de material no controlada • Desperdicio económico significativo
5	Riesgo Operativo por Dependencia de Procesos Manuales	• Documentación física aumenta riesgo de errores • Registros en papel susceptibles a pérdida • Anula la posibilidad de alertas oportunas • No permite análisis estadístico del consumo

Fuente : Elaboracion Propia (2025)

1.2.1.2 Efectos

Tabla 2 : Efectos

N°	EFFECTOS	DESCRIPCIÓN
1	Pérdidas Económicas Significativas	• Del 15-25% del inventario anual por uso no controlado del material • Impacto directo en la rentabilidad del negocio • Costos no controlados que afectan el margen de ganancia • Pérdidas económicas cuantificables y recurrentes
2	Deterioro del Ambiente Laboral	• Manejo sin datos objetivos genera sospechas • Tensión entre el personal por falta de transparencia • Ambiente de desconfianza generalizada • Afecta la moral y productividad del equipo

3	Ineficiencia Administrativa	• Dificultad para tomar decisiones en tiempo real • Falta de datos objetivos para planificar • Tiempo desperdiciado en procesos manuales (5-10 min) • Imposibilidad de optimizar recursos
4	Pérdidas Económicas Cuantificables	• Pérdida directa de rentabilidad por uso no controlado del material • Desperdicio no controlado de material ($\pm 10-20$ cm) • Costos operativos elevados por ineficiencia • Impacto económico mensurable y significativo
5	Deterioro del Ambiente Laboral por Desconfianza	• Vínculo directo con las pérdidas económicas • Ineficiencia operativa genera frustración • Clima laboral negativo afecta la retención de personal • Círculo vicioso de desconfianza y baja productividad

Fuente : Elaboracion Propia (2025)

1.2.2 Formulación del problema

¿Como evitar el dispensado excesivo de laminas de polarizado automotriz para mejorar el control de asignación precisa de material a los empleados evitando el desperdicio y pérdidas económicas ?

1.3 OBJETIVOS: GENERAL Y ESPECÍFICOS

1.3.1 Objetivo General

Desarrollar un sistema automatizado de dispensado y control de láminas de polarizado mediante microcontrolador ESP32 y notificaciones Telegram, para mejorar la administración del inventario, evitando el desperdicio y pérdidas económicas en talleres de raybanizado.

1.3.2 Objetivos Específicos

- Elaborar modelado de negocio actual y alternativo respecto al procedimiento empleado para el control de inventario de láminas de polarizado en talleres de raybanizado.
- Diseñar e implementar módulo de gestión de usuarios con autenticación mediante códigos PIN individuales.
- Diseñar e implementar módulo de control de motores paso a paso para el dispensado preciso de material.
- Implementar el módulo de solicitud de dispensado de material desde la aplicación web del empleado.
- Desarrollar un prototipo de dispensado automático con precisión de $\pm 2\text{cm}$ y capacidad para rollos de 30 metros.
- Generar historial de transacciones.
- Gestión de empleados.
- Emitir reportes de consumo.

1.4 ENFOQUE METODOLÓGICO

El enfoque del presente trabajo es cuantitativo, debido a que se medirán variables como cantidad de material dispensado, precisión del sistema de dispensado, tiempos de respuesta en notificaciones y frecuencia de uso por usuario, las cuales se registran sistemáticamente en la base de datos SQLite diseñada para este propósito.

Por otro lado, el componente cualitativo del enfoque se centra en la evaluación de la experiencia del usuario (empleados y propietarios) y la efectividad del sistema en resolver la problemática identificada, mediante observación directa y retroalimentación de usuarios.

A su vez, se establece la matriz metodológica, misma que se emplea con la finalidad de establecer técnicas y/o herramientas a emplear durante el desarrollo del presente trabajo de investigación.

Tabla 3: Matriz Metodológica.

Objetivos Específicos	Unidad de Análisis	Tipo de Información	Herramienta o Técnica	Resultado Esperado
Elaborar modelado de negocio actual y alternativo	Control de inventario en talleres	Primaria	Diagrama de Flujo de Datos, Observación directa	Modelado de Negocio Actual y Alternativo
Diseñar e implementar módulo de gestión de usuarios	Administración de empleados	Primaria	Modelo DRA, Node.js, SQLite	Módulo de gestión de usuarios con códigos PIN
Diseñar e implementar módulo de control de motores	Sistema de dispensado y corte	Primaria	Arduino IDE, C++, Motores paso a paso	Control preciso de dispensado de material
Desarrollar prototipo funcional	Dispensador completo	Primaria	ESP32, Fritzing, CAD	Prototipo físico operativo con precisión $\pm 2\text{cm}$
Implementar interfaz web	Dashboard administrativo	Primaria	Node.js, HTML, JavaScript, SQLite	Interfaz web funcional con

				reportes en tiempo real
Realizar pruebas funcionales	Sistema integrado	Primaria	Multímetro, cronómetro, logs de sistema	Sistema validado y optimizado

Fuente: Elaboración propia, 2025.

1.4.1 Alcance Temporal

Tabla 4: Cronograma de Actividades Modalidad de Proyecto de Grado

Capítulo		SEMESTRE	5TO SEMESTRE			6TO SEMESTRE	
		DURACIÓN	1ER	2DO	3ER	1ER	PARC
		ACTIVIDAD	PARCIAL	PARCIAL	PARCIAL	PARCIAL	FINAL
I	1 . 1	Diagnóstico y Justificación					
	1 . 2	Planteamiento del Problema Técnico / Tecnológico					
	1 . 3	Objetivos: General y Específicos					
	1 . 4	Enfoque Metodológico					

I I		Marco Teórico Conceptual					
I I I	3 . 1	Pruebas Realizadas					
	3 . 2	Análisis de Resultados					
		Desarrollo de Objetivos					
I V	4 . 1	Conclusiones					
	4 . 2	Recomendaciones					
		Bibliografía (Fuentes de Información)					
		Anexos					
		Última Revisión y Visto Bueno					
		Defensa Interna					

Fuente: Elaboración propia, 2025.

CAPÍTULO II

MARCO TEÓRICO CONCEPTUAL

2.1 BASES TEÓRICAS

2.1.1 Sistemas de Control Automatizado

Los sistemas de control automatizado representan una evolución tecnológica fundamental en la industria moderna, permitiendo la gestión eficiente de procesos mediante la integración de hardware y software. Estos sistemas se caracterizan por su capacidad de operar de manera autónoma, reduciendo la intervención humana y minimizando errores operacionales. En el contexto industrial, un sistema de control automatizado se define como un conjunto integrado de componentes electrónicos, mecánicos y computacionales que trabajan coordinadamente para ejecutar tareas específicas sin supervisión constante. (Ogata, 2010)

2.1.2 Internet de las Cosas (IoT)

Los dispositivos IoT se caracterizan por su capacidad de comunicación bidireccional, procesamiento de datos en el borde (edge computing) y actuación autónoma basada en algoritmos predefinidos. La arquitectura típica de un sistema IoT comprende capas de percepción (sensores y actuadores), red (protocolos de comunicación), procesamiento (análisis de datos) y aplicación (interfaces de usuario y servicios). Esta estructura multicapa permite escalabilidad y flexibilidad, adaptándose a diversos escenarios de aplicación desde pequeños sistemas embebidos hasta infraestructuras industriales complejas. (J, R, Marusic, & Palaniswami, 2013)

2.2 HERRAMIENTAS DE HARDWARE

2.2.1 Microcontrolador ESP32

El ESP32 es un microcontrolador de bajo costo y bajo consumo energético desarrollado por Espressif Systems, diseñado específicamente para aplicaciones de Internet de las Cosas. Este sistema en chip (SoC) integra capacidades de procesamiento dual-core, conectividad inalámbrica WiFi y Bluetooth, múltiples periféricos y una arquitectura robusta que lo posiciona como una solución ideal para proyectos de automatización industrial. El microcontrolador incorpora dos núcleos Xtensa LX6 de 32 bits que operan a frecuencias de hasta 240 MHz, proporcionando poder computacional suficiente para ejecutar algoritmos

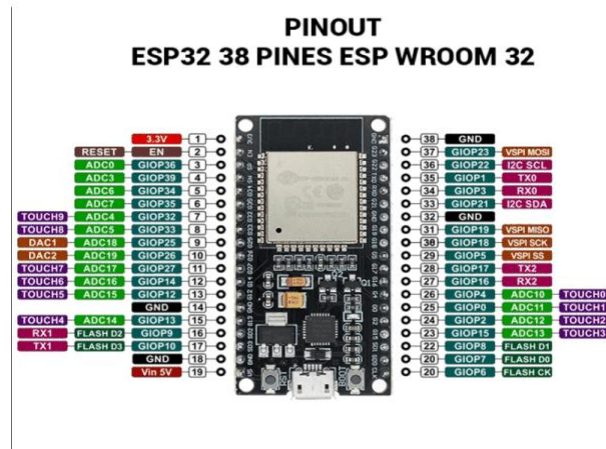
complejos y manejar múltiples tareas simultáneamente. (Systems, ESP32 Series Datasheet, 2023)

2.2.1.1 Especificaciones Técnicas del ESP32

Las especificaciones técnicas del ESP32 lo caracterizan como una plataforma robusta para desarrollo de sistemas embebidos conectados:

- Procesador: Dual-core Xtensa LX6 de 32 bits, frecuencia ajustable de 80 MHz a 240 MHz
- Memoria: 520 KB SRAM, 448 KB ROM, soporte para flash externa hasta 16 MB
- Conectividad: WiFi 802.11 b/g/n (2.4 GHz), Bluetooth v4.2 BR/EDR y BLE
- GPIO: 34 pines programables con soporte para diferentes funciones periféricas
- ADC: 18 canales de 12 bits con atenuación programable
- DAC: 2 canales de 8 bits para generación de señales analógicas
- PWM: 16 canales independientes con resolución configurable
- Interfaces: 4x SPI, 2x I2C, 2x I2S, 3x UART, 1x CAN, sensor táctil capacitivo
- Seguridad: Encriptación acelerada por hardware (AES, SHA-2, RSA), arranque seguro
- Consumo: Modo activo ~160-260 mA, modo sleep profundo ~10 μ A
- Voltaje de operación: 2.2V a 3.6V con regulador interno de 3.3V
- Temperatura: Rango operacional de -40°C a +125°C

Figura 2:Microcontrolador ESP32



Fuente: (Systems, ESP32 Series Datasheet, 2023)

2.2.2 Motor Paso a Paso 28BYJ-48

Los motores paso a paso 28BYJ-48 representan una categoría de actuadores electromecánicos de bajo costo ampliamente utilizados en aplicaciones de automatización y control de movimiento de pequeña escala. Este tipo de motor es conocido por su precisión en movimientos angulares y su facilidad de integración con plataformas de desarrollo como Arduino. El modelo 28BYJ-48 funciona mediante una secuencia de pulsos eléctricos que permiten el desplazamiento controlado del eje en pasos discretos, proporcionando un posicionamiento preciso sin necesidad de sensores de retroalimentación complejos. Debido a su tamaño compacto y bajo consumo energético, es frecuentemente empleado en proyectos educativos, robótica y prototipos automatizados. (Association, 2016)

2.2.2.1 Especificaciones Técnicas

Las especificaciones técnicas del motor paso a paso 28BYJ-48 utilizado en este proyecto son:

- Tipo de motor: Paso a paso unipolar de 4 fases
- Voltaje nominal: 5V DC
- Reducción interna: 1:64 mediante caja de engranajes
- Ángulo de paso: $5.625^\circ/64$
- Pasos por revolución: Aproximadamente 4096 pasos por vuelta completa
- Configuración de cables: 5 cables de conexión
- Corriente de operación: Aproximadamente 200-240 mA
- Torque de salida: 300-1200 gf·cm
- Frecuencia de funcionamiento: Hasta 100 Hz
- Diámetro del motor: 28 mm
- Longitud del eje: Aproximadamente 10 mm
- Peso: Aproximadamente 35-40 gramos
- Temperatura operativa: -10°C a $+50^\circ\text{C}$
- Controlador compatible: ULN2003 para interfaz con microcontroladores Arduino y similares

Figura 3: Motor Paso a Paso 28BYJ-48

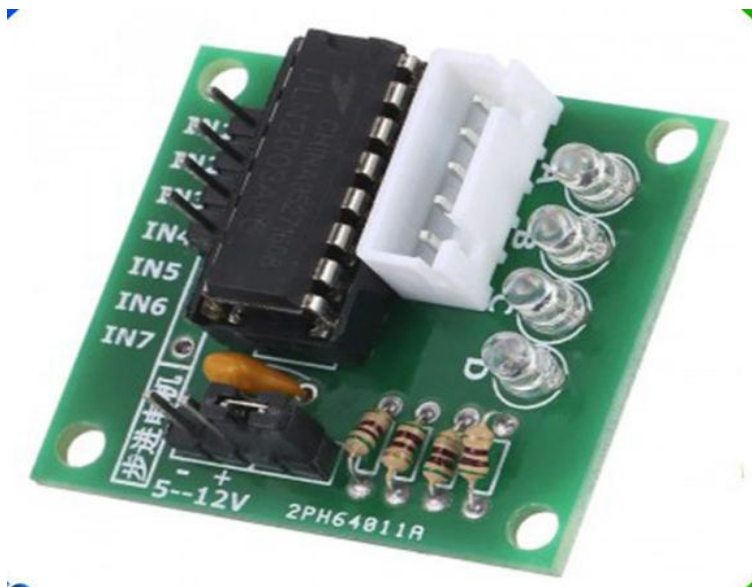


Fuente: (Association, 2016)

2.2.3 Driver ULN2003

El ULN2003 es un controlador (driver) de motor paso a paso control de fases con traductor integrado, diseñado específicamente para controlar motores bipolares. Este circuito integrado desarrollado por Allegro Microsystems proporciona una solución completa para el control de motores paso a paso, incluyendo regulación de corriente, protección térmica y capacidad de control de fases hasta 1/16 de paso. El driver opera como interfaz entre las señales lógicas de baja potencia del microcontrolador y las corrientes elevadas requeridas por las bobinas del motor, manejando hasta 2A por bobina con picos de 2.5A. (MicroSystems, 2020)

Figura 4: Driver ULN2003



Fuente: (MicroSystems, 2020)

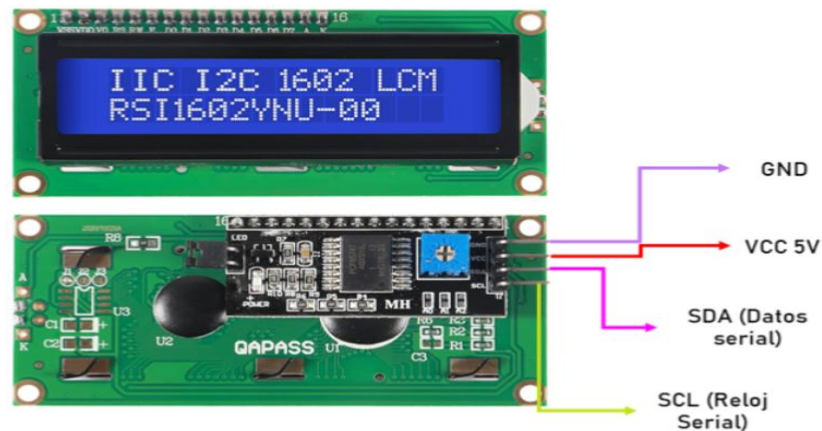
2.2.4 Pantalla LCD 20x4 con I2C

Las pantallas LCD (Liquid Crystal Display) de caracteres 20x4 con interfaz I2C constituyen dispositivos de visualización ampliamente utilizados en sistemas embebidos debido a su simplicidad de uso y bajo costo. Estas pantallas pueden mostrar 32 caracteres alfanuméricos organizados en dos filas de 16 caracteres cada una, proporcionando una interfaz de usuario

básica pero efectiva para aplicaciones de control y monitoreo. La tecnología LCD de matriz de puntos utiliza cristales líquidos que modifican su orientación bajo campos eléctricos, controlando el paso de luz a través de cada carácter formado por una matriz típica de 5x8 píxeles.

La integración de un módulo I2C (Inter-Integrated Circuit) convierte la interfaz paralela tradicional de la pantalla LCD, que requiere 6-8 pines de conexión, en una interfaz (Monje, EP Jr., 2016)

Figura 5 :Pantalla LCD 20x4 con I2C

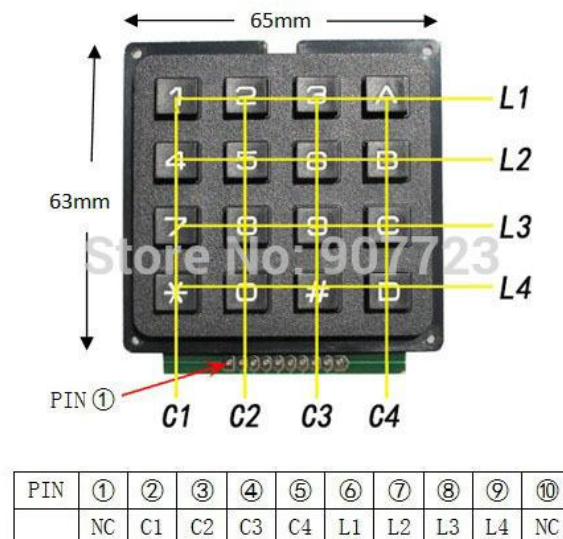


Fuente: (Royce, 1970)

2.2.5 Teclado Matricial 4x4

El teclado matricial 4x4 es un dispositivo de entrada que organiza 16 teclas en una matriz de 4 filas por 4 columnas, proporcionando una interfaz de usuario compacta y económica para sistemas embebidos. Este tipo de teclado implementa un esquema de escaneo matricial que reduce significativamente el número de conexiones requeridas comparado con teclas individuales: en lugar de necesitar 16 pines para 16 teclas independientes, la configuración matricial requiere solo 8 pines (4 filas + 4 columnas). El principio de operación se basa en la detección de conexiones entre filas y columnas cuando una tecla específica es presionada, estableciendo un camino conductivo único para cada posición de tecla. (Barrett, 2013)

Figura 6 : Teclado Matricial 4x4



Fuente: (Barrett, 2013)

2.3 HERRAMIENTAS DE SOFTWARE

2.3.1 Arduino IDE

Arduino IDE (Integrated Development Environment) es un entorno de desarrollo de código abierto diseñado específicamente para programación de microcontroladores y sistemas embebidos. Esta plataforma proporciona herramientas completas para escribir, compilar y cargar firmware en dispositivos compatibles con Arduino, incluyendo el ESP32 mediante la instalación de paquetes de soporte adicionales. El IDE implementa un editor de código con resaltado de sintaxis, un compilador basado en GCC, bibliotecas extensivas de funciones predefinidas y un sistema de gestión de bibliotecas que simplifica la incorporación de código de terceros.

La filosofía de diseño de Arduino IDE prioriza la accesibilidad y simplicidad, ocultando la complejidad inherente de la programación de microcontroladores detrás de abstracciones de alto nivel. El lenguaje de programación utilizado está basado en C/C++ con extensiones específicas del framework Arduino que proporcionan funciones simplificadas para control de pines GPIO (Arduino, 2024).

2.3.1.1 Ventajas de Arduino IDE

Las ventajas de utilizar Arduino IDE para el desarrollo del firmware del ESP32 incluyen:

- Curva de aprendizaje reducida: La interfaz simplificada y las abstracciones de alto nivel permiten a desarrolladores sin experiencia profunda en sistemas embebidos crear aplicaciones funcionales rápidamente.
- Amplia biblioteca de ejemplos: El IDE incluye numerosos ejemplos de código que demuestran funcionalidades básicas y avanzadas, sirviendo como punto de partida para proyectos personalizados.
- Comunidad activa: La extensa comunidad de usuarios Arduino proporciona soporte, tutoriales y soluciones a problemas comunes, facilitando la resolución de desafíos técnicos.
- Compatibilidad multiplataforma: Arduino IDE opera en Windows, macOS y Linux, garantizando flexibilidad en el entorno de desarrollo independientemente del sistema operativo utilizado.
- Gestión simplificada de bibliotecas: El administrador de bibliotecas integrado facilita la búsqueda, instalación y actualización de componentes de software sin manipulación manual de archivos.
- Código abierto: La naturaleza open-source del IDE permite personalizaciones, extensiones y transparencia completa del proceso de compilación

Figura 7 : Arduino IDE



Fuente: (Arduino, 2024).

2.3.2 Visual Studio Code

Visual Studio Code es un editor de código fuente desarrollado por Microsoft que ha ganado popularidad como alternativa avanzada a IDEs tradicionales. Aunque originalmente diseñado para desarrollo web, VS Code se ha convertido en una herramienta versátil para múltiples lenguajes y plataformas mediante su sistema de extensiones. La arquitectura del editor está construida sobre Electron, permitiendo operación consistente en Windows, macOS y Linux. El diseño minimalista pero poderoso proporciona funcionalidades avanzadas como resaltado de sintaxis inteligente, autocompletado basado en IntelliSense, depuración integrada y control de versiones con Git.

Para desarrollo web como el implementado en el dashboard del proyecto de dispensación, VS Code ofrece soporte nativo excelente para HTML, CSS, JavaScript y PHP. El editor proporciona validación de sintaxis en tiempo real, formateo automático de código, navegación inteligente entre archivos y sugerencias contextuales que aceleran el desarrollo. (Microsoft, Informe Anual , 2023)

Figura 8 : Visual Studio Code



Fuente: (Microsoft, Informe Anual , 2023)

2.3.3 PlatformIO

PlatformIO es un ecosistema profesional de desarrollo para sistemas embebidos que proporciona herramientas avanzadas de gestión de proyectos, compilación, depuración y testing. A diferencia de Arduino IDE, PlatformIO implementa un enfoque moderno basado en gestión de dependencias declarativa, configuración reproducible y soporte para múltiples frameworks y plataformas desde una única interfaz. La herramienta puede utilizarse mediante línea de comandos o integrada como extensión en editores populares como Visual Studio Code, Atom o CLion, proporcionando flexibilidad en la elección del entorno de desarrollo.

PlatformIO soporta más de 50 plataformas de desarrollo incluyendo AVR, ARM, ESP32, RISC-V y múltiples frameworks como Arduino, ESP-IDF, mbed y FreeRTOS. La herramienta proporciona compilación incremental optimizada que reduce significativamente tiempos de compilación en proyectos grandes. (PlatformIO, 2023)

Figura 9 : PlatformIO



Fuente: (PlatformIO, 2023)

2.4 HERRAMIENTA DE GESTIÓN DE BASE DE DATOS

2.4.1 SQLite

SQLite es un sistema de gestión de bases de datos relacional embebido, de código abierto y dominio público, ampliamente reconocido por su ligereza y facilidad de integración. A diferencia de sistemas cliente-servidor como MySQL o PostgreSQL, SQLite opera directamente sobre un único archivo en disco, eliminando la necesidad de instalar o configurar un servidor de base de datos separado. Es el motor de base de datos más implementado del mundo, presente en sistemas operativos, navegadores web, dispositivos móviles y aplicaciones embebidas. SQLite implementa la mayor parte del estándar SQL92, soportando transacciones ACID, claves foráneas, índices, vistas y disparadores. Su módulo nativo en Node.js (node:sqlite, disponible desde v22.5) permite acceso sincrónico sin dependencias externas, garantizando máxima compatibilidad y rendimiento en entornos de escritorio y servidor ligero. (SQLite Consortium, 2024)

Figura 10 : SQLite



Fuente: (SQLite Consortium, 2024)

2.4.2 DB Browser for SQLite

DB Browser for SQLite (DB4S) es una herramienta visual de código abierto diseñada para la creación, edición y exploración de bases de datos SQLite sin necesidad de escribir comandos

SQL. Proporciona una interfaz gráfica intuitiva equivalente a phpMyAdmin pero orientada a bases de datos de archivo local. Permite crear y modificar tablas, índices y columnas mediante asistentes visuales; ejecutar consultas SQL con resaltado de sintaxis; y explorar el contenido de las tablas con filtros y ordenamiento. Su naturaleza multiplataforma (Windows, macOS, Linux) y su distribución portable lo hacen ideal para entornos de desarrollo sin infraestructura de servidor. (DB Browser for SQLite, 2024)

DB Browser for SQLite implementa funcionalidades avanzadas como importación y exportación de datos en formatos CSV y SQL, editor de registros en línea, visualización de la estructura de la base de datos en árbol jerárquico, ejecución de scripts SQL por lotes, y soporte para cifrado de bases de datos mediante la extensión SQLCipher. Para el proyecto de dispensación, DB4S permite inspeccionar directamente el archivo dispensadora.db, verificar transacciones registradas, modificar configuraciones de alertas y validar la integridad de los datos sin necesidad de servidor adicional, sustituyendo completamente las funciones que antes cumplía phpMyAdmin en el servidor Node.js. (DB Browser for SQLite, 2024)

Figura 11 : DB Browser for SQLite



Fuente: (DB Browser for SQLite, 2024)

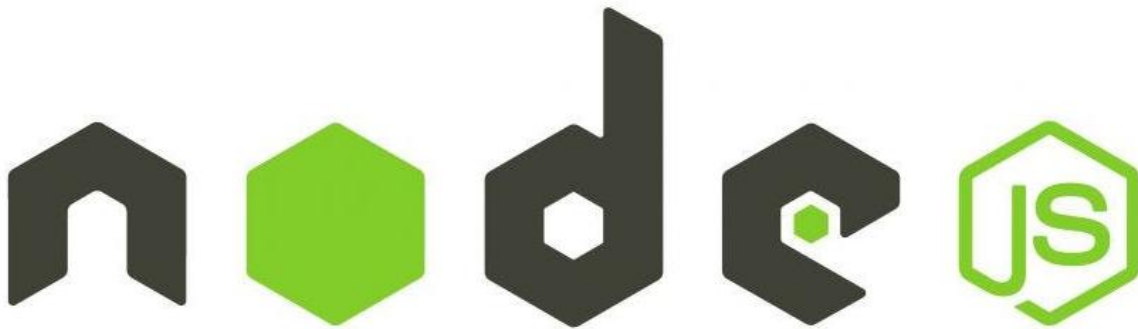
2.5 HERRAMIENTAS DE DESARROLLO WEB

2.5.1 Node.js

Node.js es un entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8 de Google Chrome. A diferencia de PHP, que sigue un modelo de ejecución por solicitud, Node.js utiliza un modelo de entrada/salida no bloqueante orientado a eventos que permite manejar miles de conexiones concurrentes con mínimo consumo de recursos. Su gestor de paquetes npm (Node Package Manager) ofrece acceso al ecosistema de librerías de código abierto más grande del mundo. Node.js incorpora desde la versión 22.5 el módulo nativo `node:sqlite` con la clase `DatabaseSync`, permitiendo acceso sincrónico a bases de datos SQLite sin dependencias externas ni compilación nativa, lo que simplifica significativamente el despliegue en entornos Windows. (OpenJS Foundation, 2024)

Express.js es el framework web minimalista y flexible para Node.js que gestiona el enrutamiento HTTP, los middlewares, la gestión de sesiones y el renderizado de plantillas EJS en el sistema de dispensación. Express implementa el patrón MVC mediante rutas declarativas (`router.get`, `router.post`) que mapean endpoints REST a controladores, reemplazando la estructura de archivos PHP con una API organizada por recursos (`/api/empleados`, `/api/transacciones`, `/api/configuración`, `/api/esp32`). El servidor Express inicializa la base de datos SQLite al arrancar (`npm start`), sirve las vistas EJS compiladas y expone los endpoints tanto para el panel web como para el microcontrolador ESP32. (OpenJS Foundation, 2024)

Figura 12 : Node.js + Express.js



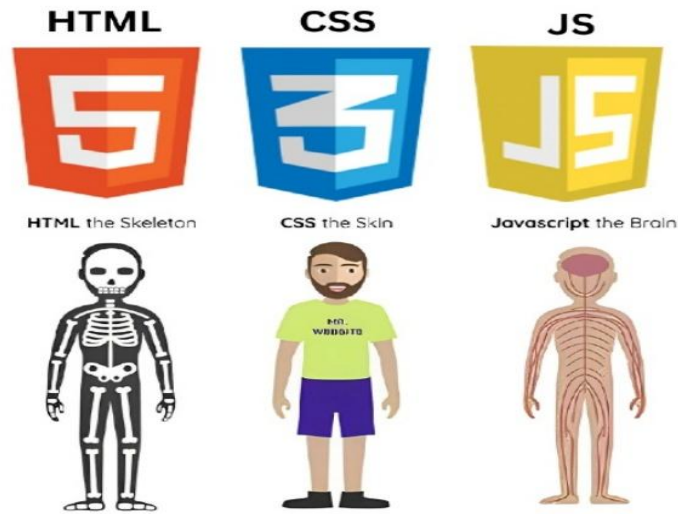
Fuente: (OpenJS Foundation, 2024).

2.5.2 HTML, CSS y JavaScript

HTML (HyperText Markup Language), CSS (Cascading Style Sheets) y JavaScript constituyen las tres tecnologías fundamentales del desarrollo web frontend. HTML proporciona la estructura y contenido semántico de páginas web mediante un sistema de etiquetas que definen elementos como encabezados, párrafos, enlaces, imágenes, tablas y formularios. HTML5, la versión actual del estándar, introduce elementos semánticos como `<header>`, `<nav>`, `<article>`, `<section>` y `<footer>` que mejoran la accesibilidad y comprensión del contenido tanto para navegadores como para motores de búsqueda. Los formularios HTML proporcionan controles de entrada como campos de texto, botones, checkboxes y selectores que permiten interacción del usuario.

JavaScript agrega interactividad y comportamiento dinámico a páginas web, ejecutándose en el navegador del cliente para manipular el DOM (Document Object Model), responder a eventos de usuario, realizar validaciones, y comunicarse con servidores mediante AJAX sin recargar páginas (corporativo, 2023)

Figura 13 : HTML, CSS y JavaScript



Fuente: (corporativo, 2023)

2.6 HERRAMIENTAS DE SIMULACIÓN DE CIRCUITOS

2.6.1 Proteus Design Suite

Proteus Design Suite es una plataforma profesional de diseño electrónico desarrollada por Labcenter Electronics que integra captura esquemática, simulación de circuitos analógicos y digitales, diseño de PCB y simulación de microcontroladores en un entorno unificado. La característica distintiva de Proteus es su capacidad de co-simulación que permite ejecutar código de firmware real en microcontroladores virtuales mientras interactúan con componentes analógicos y digitales simulados, proporcionando validación completa de diseños antes de construcción física. Esta capacidad reduce significativamente costos y tiempo de desarrollo al identificar problemas de diseño en etapas tempranas (Labcenter Electronics, 2025)

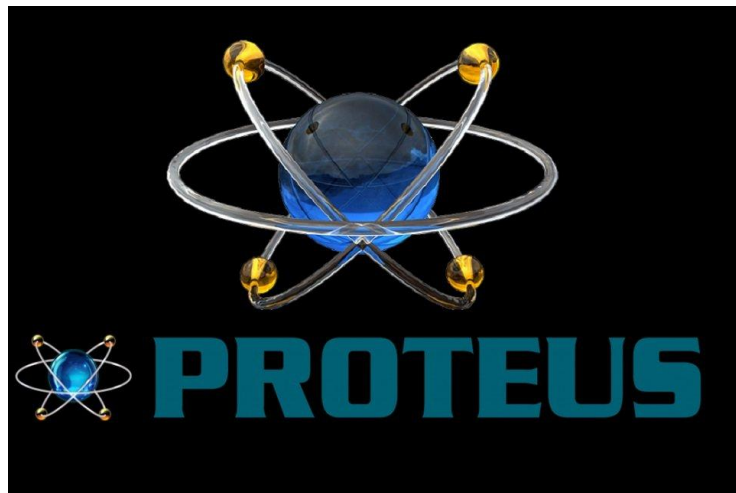
2.6.1.1 Ventajas de Proteus

Las ventajas de utilizar Proteus Design Suite en este proyecto incluyen:

- **Co-simulación de firmware:** La capacidad única de ejecutar código Arduino real en microcontroladores virtuales permite depurar lógica de control completamente en software antes de cargar firmware en hardware físico.

- Biblioteca extensa: Miles de modelos de componentes predefinidos eliminan necesidad de crear modelos personalizados para componentes comunes, acelerando diseño.
- Detección temprana de errores: La simulación identifica problemas de diseño como conexiones incorrectas, valores de componentes inadecuados o conflictos de señales antes de inversión en componentes físicos.
- Instrumentación virtual: Osciloscopios, analizadores lógicos, generadores de señales y multímetros virtuales permiten análisis detallado de señales sin equipamiento físico costoso.
- Documentación automática: Generación automática de netlists, listas de materiales (BOM) y documentación técnica facilita comunicación con fabricantes y mantenimiento futuro.

Figura 14: Proteus Design Suite



Fuente: (Labcenter Electronics, 2025)

2.7 PROTOCOLOS DE COMUNICACIÓN

2.7.1 Protocolo HTTP/HTTPS

El Protocolo de Transferencia de Hipertexto (HTTP) es el protocolo de comunicación fundamental que sustenta el funcionamiento de la World Wide Web, definiendo reglas para intercambio de información entre clientes y servidores. HTTP opera sobre el modelo petición-

respuesta donde el cliente (típicamente un navegador o dispositivo IoT) envía una solicitud especificando un método (GET, POST, PUT, DELETE), encabezados y opcionalmente un cuerpo con datos, y el servidor responde con un código de estado, encabezados y contenido solicitado (Fielding & Káiser, Estilos arquitectónicos y el diseño de arquitecturas de software basadas en redes, 1999)

2.7.2 API REST

REST (Representational State Transfer) es un estilo arquitectónico para diseño de APIs web que define principios para creación de servicios escalables, mantenibles y interoperables. Las APIs RESTful utilizan métodos HTTP estándar y URIs (Uniform Resource Identifiers) significativos para operaciones sobre recursos, tratando cada entidad del sistema como un recurso accesible mediante una URL única. Los principios REST incluyen arquitectura cliente-servidor que separa responsabilidades, ausencia de estado donde cada petición contiene toda información necesaria sin dependencia de contexto previo, y cacheo que mejora rendimiento almacenando respuestas reutilizables. (Fielding R. T., 2000).

2.7.3 JSON

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos basado en texto que representa estructuras mediante pares clave-valor, arreglos y valores primitivos. La sintaxis de JSON deriva de notación de objetos JavaScript pero es independiente del lenguaje, soportado nativamente por prácticamente todos los lenguajes de programación modernos. Los tipos de datos JSON incluyen strings entre comillas, números sin comillas, booleanos (true/false), null, objetos delimitados por llaves con pares clave-valor separados por comas, y arreglos delimitados por corchetes con valores separados por comas. (Crockford, 2006)

2.7.4 Telegram Bot API

Telegram Bot API es una interfaz HTTP que permite a desarrolladores crear bots automatizados que interactúan con usuarios en la plataforma de mensajería Telegram. Los bots actúan como cuentas especiales operadas por software en lugar de personas, capaces

de enviar mensajes, responder comandos, procesar contenido multimedia y proporcionar interfaces interactivas mediante teclados personalizados. La API se basa en peticiones HTTPS a endpoints de Telegram especificando un token único que identifica el bot, simplificando autenticación y autorización sin implementar OAuth complejo (Telegrama, 2023)

2.8 SERVIDOR Y ENTORNO DE DESARROLLO

2.8.1 Node.js Runtime Environment

Node.js constituye el entorno de ejecución principal del sistema de dispensación, reemplazando la dependencia de Apache y PHP mediante un único proceso ligero. El proyecto dispensadora-node se inicia con el comando `npm start` desde el directorio `C:\dispensadora-node`, levantando un servidor Express en el puerto 3000 que atiende tanto el panel de administración web como los endpoints del microcontrolador ESP32. La base de datos SQLite se almacena en el archivo `C:\dispensadora-node\database\dispensadora.db`, generado automáticamente en el primer arranque mediante el módulo `init-db.js` que define el esquema completo de tablas. Este enfoque simplifica el entorno de desarrollo y producción a una sola dependencia: Node.js v22 o superior. (OpenJS Foundation, 2024)

2.8.2 Express.js Framework

Express.js es un framework web minimalista para Node.js que proporciona la capa de servidor HTTP del sistema, reemplazando al servidor Apache de Node.js. A diferencia de Apache, que requiere configuración mediante archivos `httpd.conf` y módulos cargables, Express se configura directamente en código JavaScript mediante middlewares encadenados. Express gestiona el enrutamiento mediante métodos declarativos (`app.get`, `app.post`, `router.use`) que organizan los endpoints por recurso, aplica middlewares de sesión (`express-session`), parseo de cuerpos (`express.json`, `express.urlencoded`), y sirve archivos estáticos desde la carpeta `public/`. La arquitectura de Express es modular: cada grupo de rutas se define en archivos separados bajo `src/routes/`, mejorando la mantenibilidad respecto a múltiples archivos PHP. (OpenJS Foundation, 2024)

La configuración del servidor Express se centraliza en `server.js`: el puerto se define mediante la variable de entorno `PORT` (por defecto 3000), el motor de plantillas se establece como EJS (`app.set("view engine", "ejs")`), y la inicialización de la base de datos SQLite se ejecuta sincrónicamente al arrancar mediante `initDB()`. Express registra automáticamente cada solicitud con método HTTP, ruta y código de respuesta, facilitando la depuración. Para el ESP32, Express expone los endpoints `/api/validar_codigo` y `/api/registrar_transaccion` aceptando datos `application/x-www-form-urlencoded`, garantizando compatibilidad total con el código Arduino existente sin modificaciones adicionales. (OpenJS Foundation, 2024)

2.8.3 Postman

Postman es una plataforma colaborativa para desarrollo, testing y documentación de APIs que simplifica el proceso de enviar peticiones HTTP a endpoints, examinar respuestas y automatizar validaciones. La aplicación proporciona interfaz gráfica intuitiva donde desarrolladores especifican

Para el proyecto de dispensación, Postman permite validar funcionamiento de endpoints de la API REST sin necesidad de programar cliente completo. Los desarrolladores pueden simular peticiones del ESP32 enviando datos de transacciones, verificar respuestas del servidor, y diagnosticar problemas antes de integrar con firmware real. La funcionalidad de mock servers permite simular comportamiento de API durante desarrollo del firmware cuando el backend aún no está completo. La capacidad de generar snippets de código en múltiples lenguajes facilita implementación de clientes API basándose en requests probados (Postmasn INC, 2023)

2.9 METODOLOGÍA DE DESARROLLO DE SOFTWARE

2.9.1 Modelo DRA

El Modelo DRA (Desarrollo Rápido de Aplicaciones) constituye una metodología de desarrollo de software orientada a la entrega ágil de sistemas funcionales en plazos reducidos, priorizando la construcción de prototipos y la participación activa de los usuarios sobre la documentación exhaustiva. Propuesto por James Martin en la década de 1990 como

evolución de los enfoques tradicionales, el modelo organiza el trabajo en fases concurrentes que integran modelado de gestión, modelado de datos y modelado de procesos. La característica distintiva del DRA radica en su ciclo iterativo de retroalimentación, donde cada módulo se construye, revisa y refina en colaboración con los involucrados, reduciendo significativamente los tiempos de entrega respecto a metodologías lineales.

La fase de análisis de requisitos captura y documenta exhaustivamente necesidades funcionales y no funcionales del sistema mediante entrevistas con stakeholders, análisis de procesos existentes y especificación de casos de uso. Los requisitos documentados sirven como contrato entre desarrolladores y clientes, estableciendo alcance y funcionalidades esperadas. La fase de diseño traduce requisitos en arquitectura de software detallada, definiendo componentes del sistema, interfaces entre módulos, estructuras de datos, esquemas de base de datos y flujos de control. Los diagramas UML como casos de uso, clases, secuencia y despliegue documentan el diseño (Royce, 1970)

2.9.2 Fases del Modelo DRA

Las fases específicas aplicadas al desarrollo del sistema de dispensación automatizada son:

Fase 1: Análisis de Requisitos. Se identifican necesidades del sistema incluyendo dispensación precisa de material mediante códigos de empleado, registro de transacciones en base de datos, dashboard web para monitoreo, notificaciones automáticas y control de inventario. Los requisitos no funcionales especifican precisión de dispensación, tiempo de respuesta, seguridad de datos y disponibilidad del sistema.

Fase 2: Diseño del Sistema. Se define la arquitectura cliente-servidor con el ESP32 como cliente de campo, servidor web Node.js con Express, base de datos SQLite y bot de Telegram. Los diagramas UML modelan los casos de uso (verificar empleado, dispensar material, consultar historial), diagramas de clases para las entidades principales, diagramas de secuencia para los flujos de transacciones y el modelo entidad-relación para la base de datos.

Fase 3: Implementación. El desarrollo integra el firmware del ESP32 en Arduino IDE para el control de hardware y la comunicación WiFi, el servidor Node.js con Express para la lógica de backend y la API REST, las vistas HTML/CSS/JavaScript con motor de plantillas EJS para el

dashboard administrativo, los esquemas SQLite para las tablas de la base de datos y la configuración del bot de Telegram para las notificaciones automáticas.

Fase 4: Pruebas. Testing unitario verifica funciones individuales como lectura de teclado, control de motor y queries de base de datos. Testing de integración valida comunicación ESP32-servidor, servidor-base de datos y servidor-Telegram. Testing de sistema evalúa flujo completo desde ingreso de código hasta registro de transacción y notificación.

Fase 5: Despliegue. Se ensambla hardware final, se carga firmware en ESP32, se configura servidor web con dominio/IP accesible, se inicializa base de datos con datos de empleados, y se entrena usuarios en operación del sistema.

Fase 6: Mantenimiento. Soporte continuo incluye corrección de bugs reportados, optimización de consultas de base de datos, actualización de firmware para nuevas características, respaldos periódicos de datos y monitoreo de disponibilidad del sistema.

Figura 15:Modelo DRA



Fuente: (Royce, 1970)

2.10 METODOLOGÍA DE DESARROLLO DE HARDWARE

2.10.1 Prototipado Iterativo

El prototipado iterativo es una metodología de desarrollo de hardware que enfatiza construcción incremental mediante ciclos repetidos de diseño, construcción, prueba y refinamiento. A diferencia del desarrollo lineal, esta aproximación reconoce que requisitos de hardware frecuentemente evolucionan durante desarrollo y que problemas de diseño emergen solo mediante experimentación física. Cada iteración produce un prototipo funcional con complejidad creciente, permitiendo validar conceptos, identificar problemas mecánicos o eléctricos tempranamente, y ajustar diseño basándose en testing real.

Las ventajas del prototipado iterativo incluyen detección temprana de fallos de diseño cuando correcciones son menos costosas, flexibilidad para incorporar mejoras descubiertas durante testing, y validación continua con stakeholders que proporciona feedback. Cada iteración permite aprender de errores sin comprometer cronograma completo del proyecto. Para el sistema de dispensación, las iteraciones incluyen prototipo breadboard probando ESP32 y motor, prototipo con PCB perforada integrando drivers y periféricos, y versión final con PCB diseñada y chasis mecánico completo (Ulrich & Eppinger, 2015)

2.10.2 Fases de Desarrollo de Hardware

El desarrollo de hardware del sistema de dispensación sigue fases estructuradas:

Fase 1: Especificación de Requisitos. Se definen requisitos eléctricos (voltajes, corrientes, señales), mecánicos (dimensiones, torques, movimientos), y de interfaz (pantalla, teclado, LEDs). Los requisitos incluyen precisión de dispensación $\pm 2\text{mm}$, velocidad de dispensación 10cm/s , consumo máximo 30W , dimensiones compactas $<30 \times 30 \times 40\text{cm}$, interfaz de usuario simple con pantalla LCD y teclado 4×4 .

Fase 2: Diseño Conceptual. Se desarrollan esquemas de bloques mostrando componentes principales y conexiones. Se seleccionan componentes específicos (ESP32, 28BYJ-48, ULN2003, LCD I2C, teclado matricial) basándose en requisitos. Se crean diagramas de flujo de señales y diagramas de potencia.

Fase 3: Diseño Detallado. Se elaboran esquemáticos eléctricos completos usando herramientas CAD como Proteus o KiCad. Se diseñan PCBs con ruteo de pistas considerando separación de señales analógicas/digitales, planos de tierra, y minimización de interferencias. Se modelan componentes mecánicos en CAD 3D definiendo chasis, soportes de motor y guías de material.

Fase 4: Prototipado y Construcción. Se ensambla prototipo inicial en protoboard validando conexiones eléctricas. Se fabrican PCBs mediante servicios de manufactura o métodos caseros. Se imprimen componentes mecánicos en 3D o fabrican mediante métodos sustractivos. Se integran todos subsistemas verificando ensamblaje correcto.

Fase 5: Testing y Validación. Se prueban subsistemas individualmente (motor, teclado, LCD, comunicación WiFi). Se valida integración completa ejecutando ciclos operacionales completos. Se miden parámetros críticos como precisión de dispensación, tiempo de respuesta y consumo energético. Se identifican y corrigen problemas de hardware o firmware.

Fase 6: Refinamiento. Se optimiza diseño basándose en resultados de testing. Se mejora ruteo de PCB reduciendo interferencias. Se ajustan parámetros mecánicos para mejor precisión. Se implementan mejoras de usabilidad basadas en feedback de usuarios.

Fase 7: Documentación. Se generan esquemáticos finales, listas de materiales (BOM), diagramas de ensamblaje, y manuales de usuario. Se documentan procedimientos de calibración, mantenimiento y troubleshooting.

2.11 HERRAMIENTAS UML (LENGUAJE DE MODELADO UNIFICADO)

2.11.1 UML

UML (Unified Modeling Language) es un lenguaje de modelado visual estandarizado utilizado para especificar, visualizar, construir y documentar artefactos de sistemas de software y otros sistemas complejos. Desarrollado por Grady Booch, Ivar Jacobson y James Rumbaugh a mediados de los 1990s, UML unificó notaciones previamente fragmentadas en un estándar

mantenido por el Object Management Group (OMG). El lenguaje proporciona conjunto rico de diagramas que capturan diferentes perspectivas del sistema: diagramas estructurales muestran organización estática, mientras diagramas de comportamiento ilustran aspectos dinámicos. (Pressman R. S., 2010)

2.11.2 Diagrama de Flujo

Un Diagrama de Flujo representa la esquematización gráfica de un algoritmo, el cual muestra gráficamente los pasos o procesos a seguir para alcanzar la solución de un problema. Su correcta construcción es sumamente importante porque, a partir del mismo se escribe un programa en algún Lenguaje de Programación. Si el Diagrama de Flujo está completo y correcto, el paso del mismo a un Lenguaje de Programación es relativamente simple y directo.

Es importante resaltar que el Diagrama de Flujo muestra el sistema como una red de procesos funcionales conectados entre sí por "Tuberías" y "Depósitos" de datos que permite describir el movimiento de los datos a través del Sistema. Este describirá: Lugares de

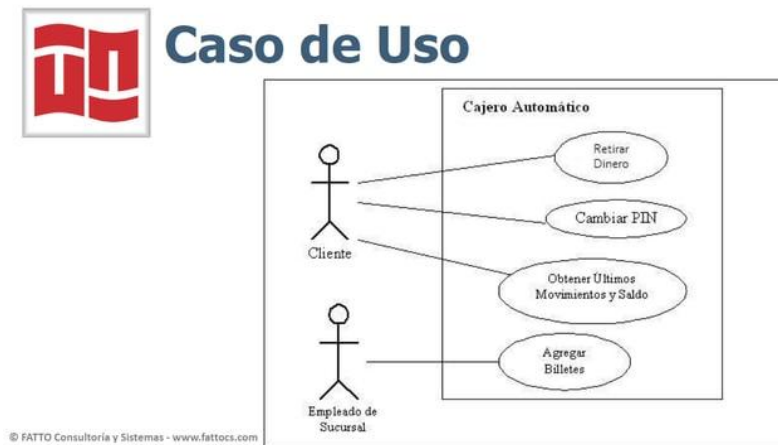
Origen y Destino de los datos, Transformaciones a las que son sometidos los datos. (Joyanes Aguilar, 2013)

Tabla 5 : Símbolos utilizados en los diagramas de flujo*Fuente : (Joyanes Aguilar, 2013)*

2.11.3 Diagramas de Casos de Uso

Los diagramas de casos de uso capturan requisitos funcionales del sistema desde perspectiva del usuario, mostrando qué puede hacer el sistema sin detallar cómo lo implementa internamente. Un caso de uso representa funcionalidad discreta que proporciona valor observable a un actor (usuario o sistema externo). Los diagramas muestran actores como figuras de palo fuera del sistema, casos de uso como óvalos dentro del límite del sistema, y relaciones entre ellos mediante líneas. Esta notación simple comunica efectivamente alcance y capacidades del sistema a stakeholders no técnicos (Cockburn, 2020).

Figura 16 : Diagrama de Caso de Uso

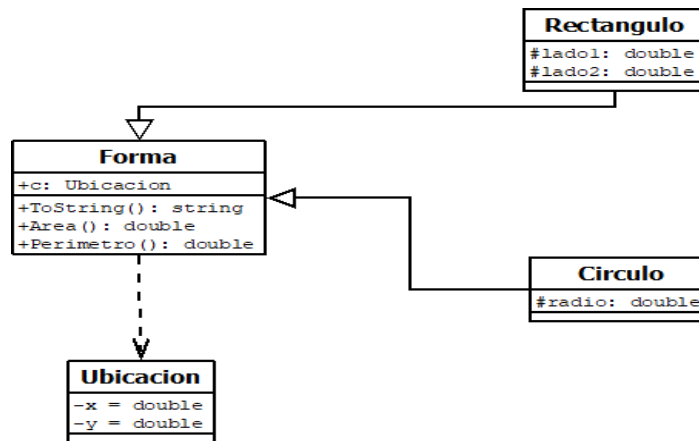


Fuente: (Cockburn, 2020).

2.11.4 Diagramas de Clases

Los diagramas de clases son el diagrama estructural fundamental de UML que modela estructura estática del sistema mostrando clases, sus atributos, métodos y relaciones entre clases. Una clase se representa como rectángulo dividido en tres compartimentos: nombre de clase (superior), atributos (medio) y métodos (inferior). Los atributos especifican tipo de datos y opcionalmente visibilidad (+ público, - privado, # protegido), mientras métodos incluyen parámetros y tipos de retorno. Este nivel de detalle permite generar código esqueleto directamente desde diagramas (Group, Object Management, 2017).

Figura 17 : Diagrama De Clases



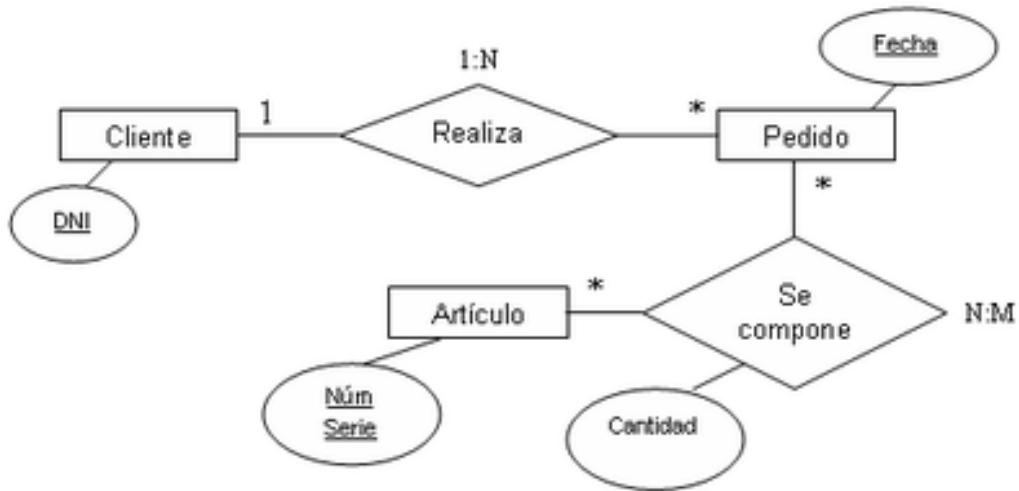
Fuente : (Group, Object Management, 2017).

2.12 MODELADO DE BASE DE DATOS

2.12.1 Diagrama Entidad-Relación (E-R)

El Diagrama Entidad-Relación es una técnica de modelado conceptual que representa estructura de bases de datos mediante entidades (objetos del mundo real), atributos (propiedades de entidades) y relaciones (asociaciones entre entidades). Desarrollado por Peter Chen en 1976, el modelo E-R abstrae detalles de implementación física, enfocándose en qué información debe almacenarse y cómo se relaciona. Las entidades se representan como rectángulos, atributos como óvalos conectados a entidades, y relaciones como rombos conectando entidades participantes. Esta notación visual facilita comunicación entre analistas de base de datos, desarrolladores y stakeholders de negocio (Chen, 1979).

Figura 18 : Diagrama Entidad-Relacion



Fuente : (Chen, 1979).

2.13 ARQUITECTURA DEL SISTEMA

La **arquitectura del sistema de dispensación automatizada** implementa un modelo distribuido de tres capas que separa responsabilidades entre dispositivos embebidos, servidor backend y capa de presentación. Esta estructura modular mejora el mantenimiento, escalabilidad y flexibilidad del sistema, integrando tecnologías IoT, aplicaciones web y control embebido en un ecosistema cohesivo. En la capa de hardware, el microcontrolador **ESP32** ejecuta firmware desarrollado en Arduino IDE, controlando periféricos como teclado matricial, pantalla LCD, driver ULN2003 para motor paso a paso **28BYJ-48** y servomotores auxiliares. El ESP32 se conecta vía WiFi al servidor backend mediante protocolo **HTTP/HTTPS**, gestionando en tiempo real la dispensación física y reduciendo la latencia.

El **servidor backend**, basado en **stack Node.js (Node.js, Express, SQLite, EJS)**, ofrece servicios de API REST para la comunicación con el ESP32 y con el **dashboard web**, desarrollado con **HTML5, CSS3, JavaScript y Bootstrap**. Este panel de administración muestra datos en tablas y gráficos **Chart.js**, permitiendo control, registro de transacciones, gestión de usuarios y configuración del sistema. La integración con **Telegram Bot API** envía alertas a administradores, mientras que la base de datos SQLite centraliza la información con confiabilidad ACID. La arquitectura modular

permite escalar el sistema fácilmente, agregar nuevos clientes o funciones sin afectar las capas existentes.

2.14 DEFINICIÓN DE TÉRMINOS BÁSICOS

API (Application Programming Interface): Conjunto de definiciones y protocolos que permite la comunicación entre diferentes componentes de software, especificando cómo los servicios deben solicitarse y qué respuestas esperar.

Actuador: Dispositivo que convierte señales de control en acción física, como motores que generan movimiento o relés que conmutan circuitos eléctricos.

Backend: Capa de servidor de una aplicación que procesa lógica de negocio, gestiona bases de datos y proporciona servicios a frontend, no directamente visible para usuarios finales.

Dashboard: Interfaz visual que consolida y presenta información crítica del sistema mediante gráficos, tablas y controles, permitiendo monitoreo y gestión centralizados.

Firmware: Software embebido en dispositivos de hardware que controla funciones de bajo nivel del dispositivo, típicamente almacenado en memoria no volátil.

Framework: Estructura de software que proporciona funcionalidades genéricas que pueden especializarse mediante código adicional del usuario, como Bootstrap para desarrollo web.

Frontend: Capa de presentación de una aplicación que interactúa directamente con usuarios, implementada típicamente con HTML, CSS y JavaScript en aplicaciones web.

GPIO (General Purpose Input/Output): Pines configurables de un microcontrolador que pueden funcionar como entradas para leer señales o salidas para controlar dispositivos externos.

I2C (Inter-Integrated Circuit): Protocolo de comunicación serial síncrono que utiliza dos líneas (SDA para datos, SCL para reloj) permitiendo conectar múltiples dispositivos en bus compartido.

IoT (Internet of Things): Paradigma donde dispositivos físicos se conectan a internet, recopilando y compartiendo datos para crear sistemas inteligentes y automatizados.

JSON (JavaScript Object Notation): Formato ligero de intercambio de datos basado en texto que utiliza sintaxis de objetos JavaScript para representar estructuras de datos.

Microcontrolador: Circuito integrado programable que contiene procesador, memoria y periféricos en un solo chip, diseñado para control de sistemas embebidos.

Control de fases: Técnica de control de motores paso a paso que subdivide cada paso completo en fracciones más pequeñas, incrementando resolución y suavidad de movimiento.

Motor Paso a Paso: Motor eléctrico que divide rotación completa en pasos discretos, permitiendo control preciso de posición sin sistema de retroalimentación.

OTA (Over-The-Air): Capacidad de actualizar firmware de dispositivos remotamente mediante conexión de red sin acceso físico al dispositivo.

PCB (Printed Circuit Board): Placa de circuito impreso que interconecta componentes electrónicos mediante pistas conductoras grabadas en sustrato no conductor.

PWM (Pulse Width Modulation): Técnica de modulación que varía ancho de pulsos en señal digital para controlar potencia promedio entregada a dispositivos, comúnmente usada para control de velocidad de motores.

REST (Representational State Transfer): Estilo arquitectónico para diseño de APIs web que utiliza métodos HTTP estándar y URIs para operaciones sobre recursos.

RSID: Identificador de revisión en documentos Word que rastrea cambios realizados por diferentes usuarios o sesiones de edición.

Servomotor: Actuador rotativo con control integrado de posición angular mediante retroalimentación, controlado por señales PWM.

SoC (System on Chip): Circuito integrado que incorpora todos componentes de un sistema computacional completo (procesador, memoria, periféricos) en un solo chip.

SQL (Structured Query Language): Lenguaje estándar para gestión y consulta de bases de datos relacionales, permitiendo crear, leer, actualizar y eliminar datos.

UML (Unified Modeling Language): Lenguaje de modelado visual estandarizado para especificar, visualizar y documentar artefactos de sistemas de software.

WiFi: Tecnología de red inalámbrica basada en estándares IEEE 802.11 que permite transmisión de datos mediante ondas de radiofrecuencia.

CAPÍTULO III

PROPUESTA DE INNOVACIÓN O

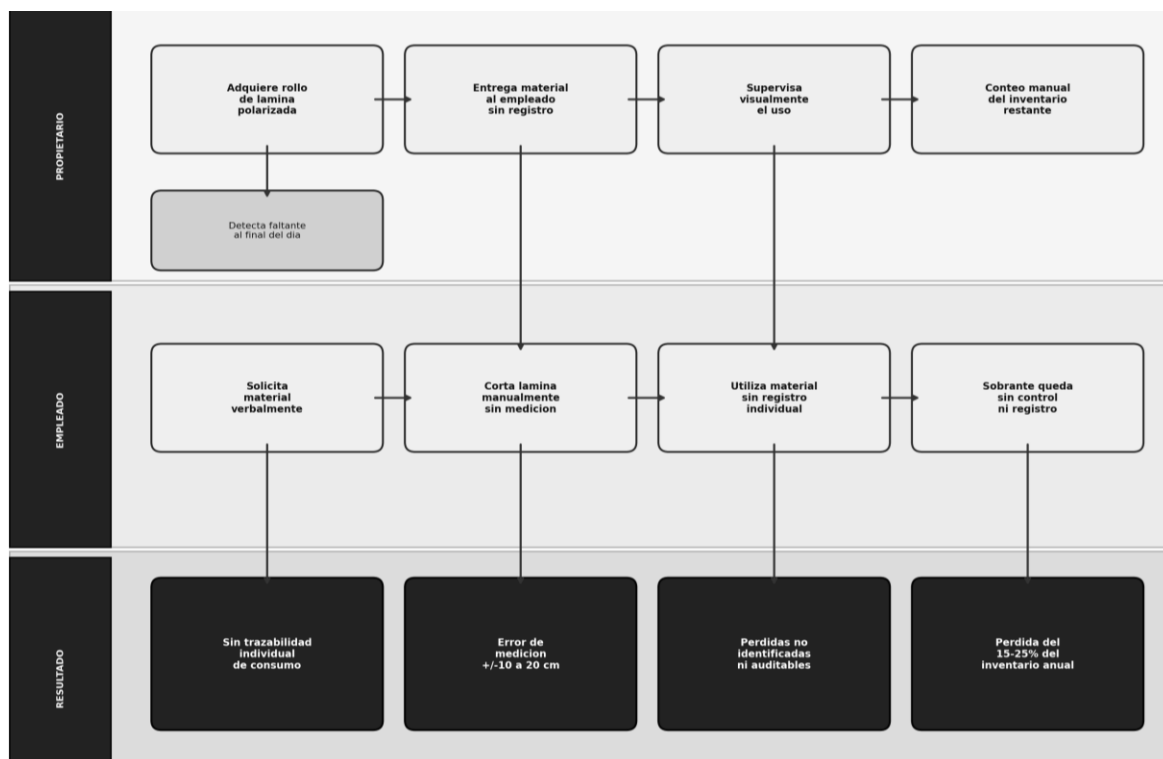
SOLUCIÓN DEL PROBLEMA

3.1 ELABORACIÓN DE MODELADO DE NEGOCIO ACTUAL Y ALTERNATIVO

3.1.1 Modelado de Negocio Actual

El modelo de negocio actual permite representar de manera gráfica la situación existente en los talleres de polarizado automotriz respecto al control y dispensado de material, evidenciando las actividades manuales, los actores involucrados y las deficiencias del proceso actual que justifican la implementación del sistema propuesto.

Figura 19 : Modelo de Negocio Actual

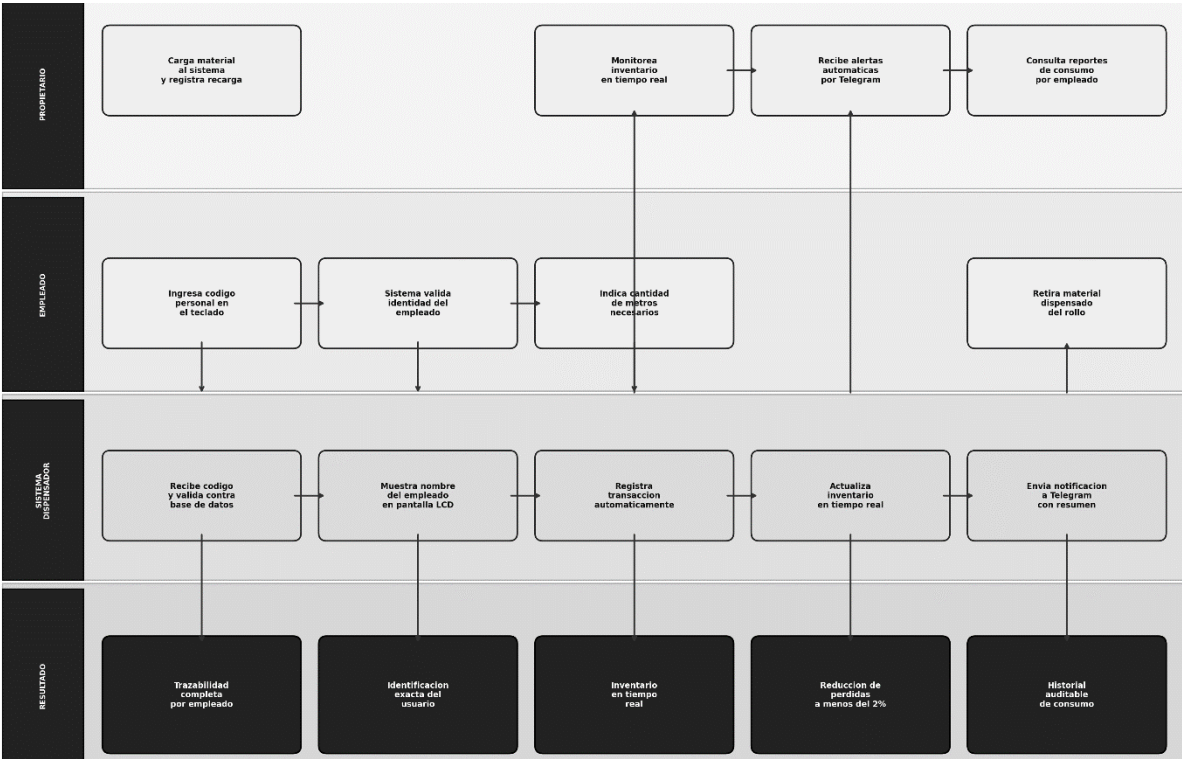


Fuente: Elaboración propia, 2025

3.1.2 Modelado de Negocio Alternativo

El modelo de negocio alternativo surge como propuesta de solución que permite visualizar el proceso mejorado mediante la automatización. En este modelo, el microcontrolador ESP32 centraliza el control del dispensado, el sistema web Node.js gestiona el inventario en tiempo real y las notificaciones Telegram informan al administrador de cada transacción, eliminando los controles manuales y registros en papel del proceso actual.

Figura 20: Modelo de Negocio Alternativo



Fuente: Elaboración propia, 2025

3.2 MODELO DRA

El Modelo DRA estructura el desarrollo del sistema en fases secuenciales donde cada etapa debe completarse antes de iniciar la siguiente. Para el presente proyecto se identificaron tres fases principales: Modelado de Gestión, Modelado de Datos y Modelado de Procesos, descritas a continuación.

3.2.1 FASE 1: MODELADO DE GESTIÓN

La Fase de Modelado de Gestión define los requerimientos del sistema mediante el análisis de las necesidades del taller de polarizado. La Tabla 6 presenta los requerimientos funcionales, que describen las capacidades operativas del sistema, y los requerimientos no funcionales, que establecen los criterios de calidad y rendimiento que cumple.

Tabla 6 : Requerimientos funcionales y no Funcionales

N°	Requerimientos Funcionales	Requerimientos No Funcionales
1	Autenticar de empleados Registrar usuarios	Tiempo de respuesta a solicitud de dispensado en un tiempo máximo 3 segundos
2	Validar de código en base de datos SQLite Asignar roles y permisos a usuarios	Autenticar código de empleado
3	Dispensar automáticamente la cantidad solicitada	Actualizar el inventario en tiempo real
4	Registrar cantidad de metros dispensados	Interfaz intuitiva

	Registrar nuevos tipos de material	
5	Registrar de empleados	Protección de datos personales de los empleados con acceso restringido al administrador
6	Enviar notificaciones vía Telegram	Configurar el Sistema por personal autorizado
7	Modificar datos de los empleados	Compatible con cualquier navegador
8	Generar reportes de dispensado de material Registrar el ingreso de materiales de polarizado	Backup automático de BD cada 30 días
9	Emitir mensajes de control de alertas de material bajo	Mostrar alertas visuales y sonoras ante errores y falta de material
10	Emitir historial completo de transacciones por fechas Actualizar automáticamente el stock después de cada dispensado	Calibración manual del dispensado del material Precisar en la entrega del material solicitado
11	Seleccionar el tipo de lámina requerida	Mostrar la cantidad disponible de cada tipo de lámina
12	Verificar la disponibilidad antes del dispensado	Verificación de disponibilidad en tiempo real, respuesta menor a 1 segundo
13	Registrar fecha, hora y usuario responsable del retiro	Trazabilidad completa e inmutable de los registros para auditoría interna
14	Consulta de movimiento de inventario	Consultas de inventario disponibles con tiempo de respuesta menor a 2 segundos

15	Generar reportes diarios, semanales y mensuales	Generación de reportes en formato PDF en menos de 5 segundos
16	Configurar límites mínimos de stock	Configuración de límites accesible únicamente por el administrador del sistema
17	Enviar alertas cuando exista escasez de materiales	Alertas enviadas en tiempo máximo de 10 segundos tras detectar escasez de material
18	Generar reportes de inventario	Reportes exportables en formato PDF, compatibles con cualquier visor de documentos
19	Conectar con sensores y dispensadores automáticos	Disponibilidad del sistema del 99% durante el horario operativo del establecimiento

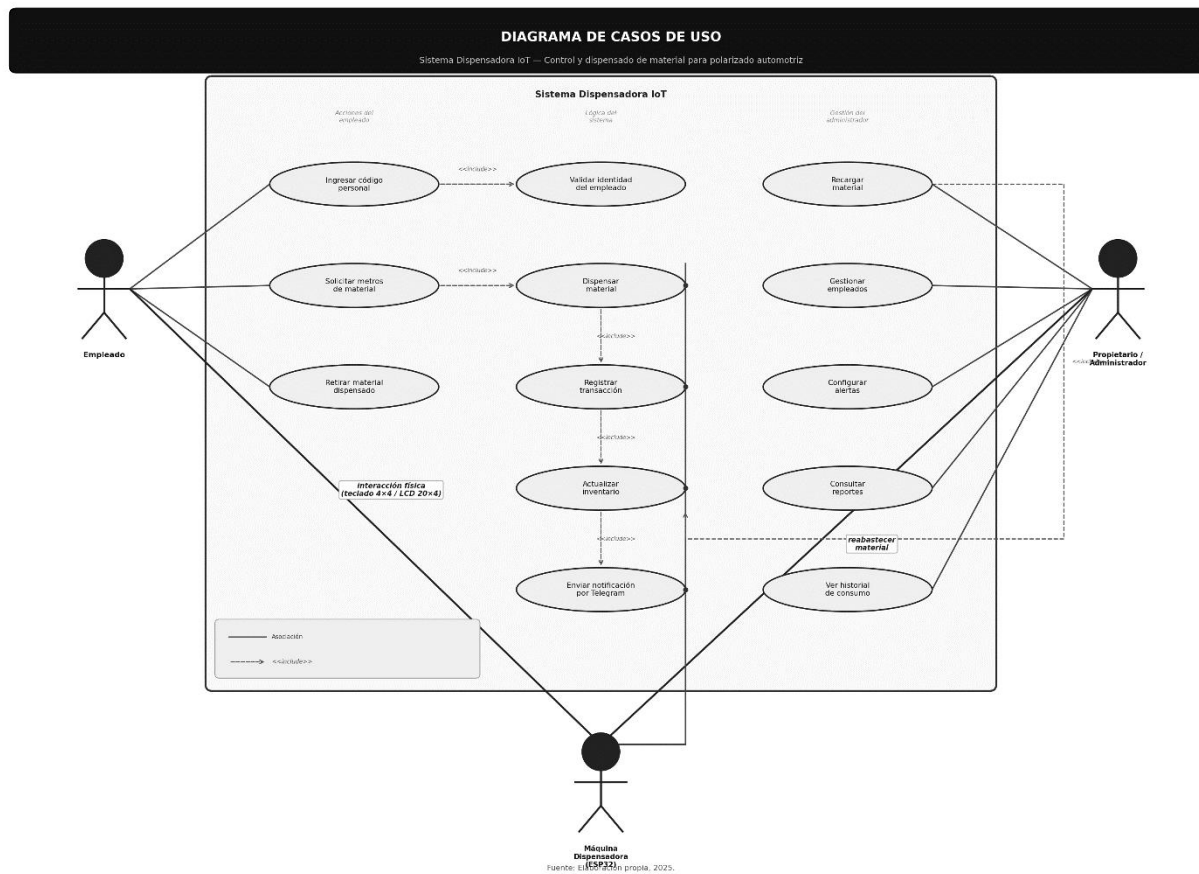
Poner como acciones verbos y usar emitir ,todas las tareas que hace el proyecto

FUENTE: Elaboración propia, 2025

3.2.1.1 Diagrama de casos de uso general

El diagrama de casos de uso representa las interacciones principales entre los tres actores del sistema (Empleado, Administrador y Sistema Dispensador) y las funcionalidades disponibles. El actor Empleado interactúa con el proceso de dispensado de material mediante código PIN; el Administrador gestiona empleados, configuración, recargas e informes desde el panel web; y el Sistema Dispensador (ESP32) ejecuta el dispensado físico del material y registra automáticamente cada transacción en la base de datos.

Figura 21 : Diagrama de Casos de usos - Sistemas Dispensadora IoT



FUENTE: Elaboración propia, 2025

3.2.1.2 Especificación de casos de uso

La especificación de casos de uso detalla el flujo de interacción entre el actor y el sistema para cada funcionalidad principal. A continuación se presentan los dos casos de uso fundamentales del sistema: Dispensar Material e Iniciar Sesión en Dashboard.

Tabla 7 : Especificación de caso de uso - Dispensar Material

Caso de Uso	Dispensar Material de Polarizado
Actores	EMPLEADO
Precondición	El empleado se encuentra registrado en el sistema con código PIN activo. Debe haber material disponible en el rollo.
Postcondición	El material es dispensado correctamente. La transacción queda registrada en la base de datos. Se actualiza el inventario.
Flujo Principal	1. El empleado ingresa su código PIN en el teclado 2. El sistema valida el código en la BD 3. El sistema muestra nombre del empleado y material disponible 4. El empleado ingresa los metros deseados (1-9m) 5. El sistema valida que haya material suficiente 6. Los motores dispensan el material solicitado 7. El sistema registra la transacción en BD 8. Se envía notificación vía Telegram 9. Se actualiza el dashboard en tiempo real

Flujo Alternativo	A1. Si el código es inválido, mostrar error y solicitar nuevamente A2. Si no hay material suficiente, mostrar alerta y denegar A3. Si hay error en motores, registrar fallo y notificar

FUENTE: Elaboración propia, 2025

Tabla 8 : Especificación de caso de uso - Iniciar Sesión Dashboard

Caso de Uso	Iniciar Sesión en Dashboard Web
Actores	ADMINISTRADOR
Precondición	El administrador cuenta con credenciales válidas (username y password).
Postcondición	El administrador accede al panel de control con todas las funcionalidades habilitadas.
Flujo Principal	1. El administrador accede a http://localhost:3000 2. Ingresa username y password 3. El sistema valida credenciales en BD 4. Se inicia sesión y se crea token de sesión

	5. Se redirige al panel principal (dashboard)
Flujo Alternativo	A1. Si las credenciales son incorrectas, mostrar error A2. Si la sesión expira, redirigir a login

FUENTE: Elaboración propia, 2025

Se documentan además los siguientes casos de uso complementarios: (CU-03) Gestionar Empleados: el Administrador puede agregar un nuevo empleado ingresando nombre y generando PIN automático, editar datos o desactivar su acceso desde el dashboard web; (CU-04) Consultar Historial: el Administrador filtra el registro de dispensados por rango de fechas o por empleado y puede exportar el reporte en formato PDF; (CU-05) Registrar Recarga de Material: el Administrador ingresa la cantidad de metros recargados actualizando el stock disponible; (CU-06) Configurar Sistema: el Administrador ajusta umbrales de alerta, credenciales del Bot de Telegram y calibración de pasos por metro del motor.

3.2.1.3 Actores del sistema

Los actores del sistema son las entidades que interactúan con él, tanto humanas como no humanas. Se identificaron tres actores principales cuyas responsabilidades son distintas y complementarias: el Empleado, el Administrador y el Sistema Dispensador (ESP32), como se detalla en la Figura 22 y la Tabla 9.



Figura 22 : Actores del Sistema



FUENTE: Elaboración propia, 2025

Tabla 9 : Tabla de Actores

Actor	Descripción	Responsabilidades
Empleado	Usuario del sistema que utiliza material de polarizado en el taller	- Autenticarse con código PIN - Solicitar material - Usar material dispensado
Administrador	Propietario o encargado del taller con acceso completo al sistema	- Gestionar empleados - Consultar reportes - Recargar material - Configurar sistema
Sistema Dispensador (ESP32)	Actor no humano que ejecuta el dispensado físico del material y registra automáticamente cada transacción en la base de datos	- Valida el código PIN ingresado - Controla el motor paso a paso para el dispensado de material - Registra la transacción via API REST - Muestra el estado del proceso en la pantalla LCD

FUENTE: Elaboración propia, 2025

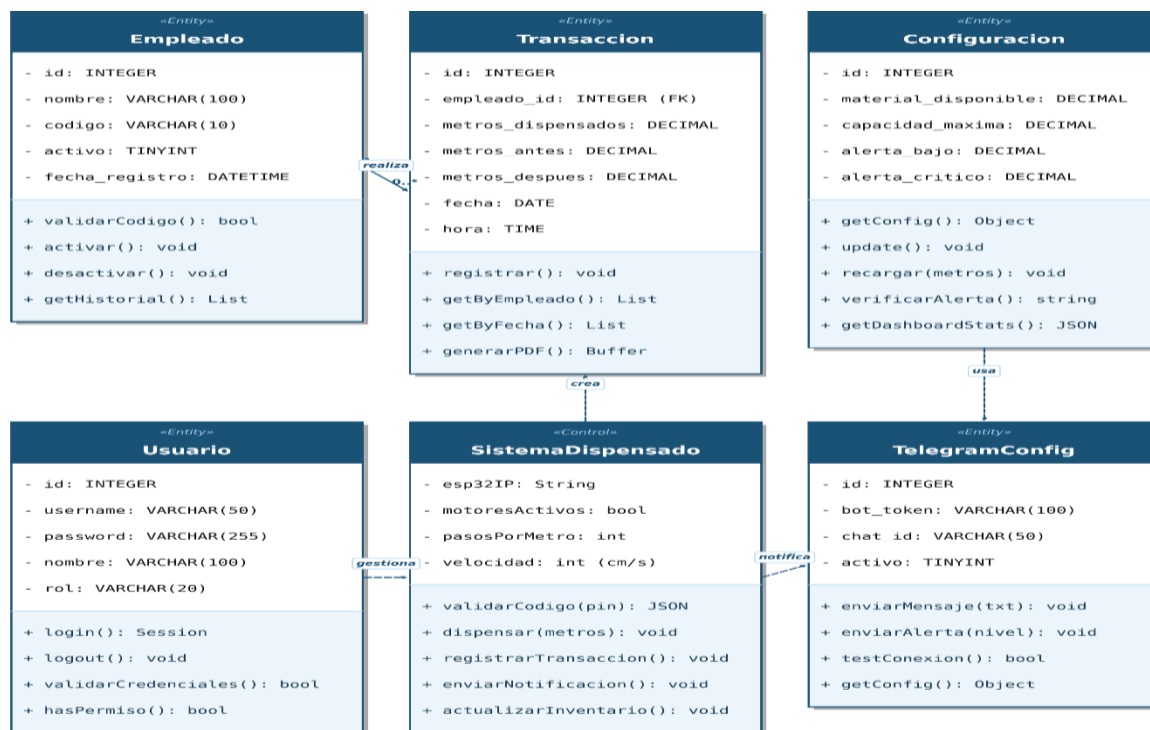
3.2.2 FASE 2: MODELADO DE DATOS

La Fase de Modelado de Datos define la estructura de almacenamiento del sistema. Se presentan el diagrama de clases, el modelo entidad-relación y el diccionario de datos que describen las cinco tablas principales de la base de datos SQLite: empleados, configuración, historial_transacciones, historial_recargas y usuarios.

3.2.2.1 Diagrama de clases

El diagrama de clases muestra la estructura orientada a objetos del sistema, representando las clases principales, sus atributos, métodos y relaciones. Las clases identificadas son: Empleado (gestión de usuarios del sistema), Transacción (registro de dispensados), Configuración (parámetros operativos del rollo), Usuario (administrador del panel web) y SistemaDispensado (control de los motores y sensores del ESP32).

Figura 23 : Diagrama de Clases-Sistema Dispensadora IoT



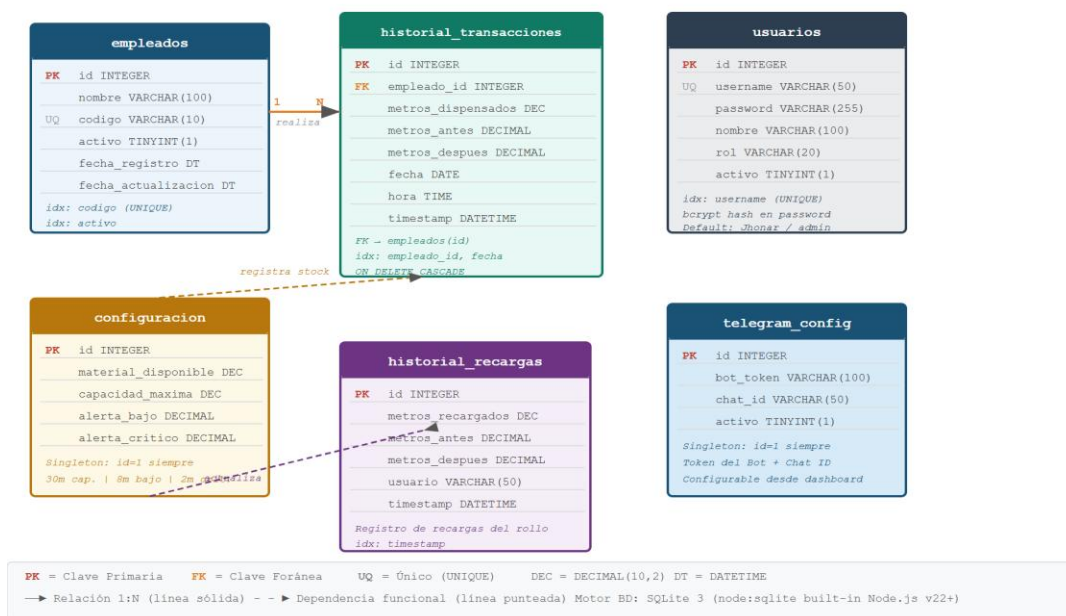
FUENTE: Elaboración propia, 2025

3.2.2.2 Diagrama de base de datos E-R

El diagrama entidad-relación (E-R) describe la estructura lógica de la base de datos SQLite del sistema. La tabla central es historial_transacciones, que se relaciona con empleados mediante clave foránea (empleado_id), y con configuración para registrar el estado del material antes y después de cada dispensado. La tabla historial_recargas registra los eventos de recarga del rollo por parte del administrador.

Figura 24: Diagrama de Entidad-Relacion-Base de Datos

Figura 25: Diagrama Entidad-Relación — Base de Datos SQLite (dispensadora.db)



FUENTE: Elaboración propia, 2025

3.2.2.3 Diccionario de datos

El diccionario de datos detalla la estructura de cada tabla de la base de datos SQLite, especificando el nombre del campo, tipo de dato, tamaño y descripción funcional. Esta documentación sirve como referencia técnica para el mantenimiento y futuras ampliaciones del sistema.

Tabla 10 : Diccionario de Datos (Empleados)

CAMPO	TIPO	TAMAÑO	DESCRIPCIÓN
id	INT	11	Identificador único del empleado (PK, auto-increment)
nombre	VARCHAR	100	Nombre completo del empleado
código	VARCHAR	10	Código PIN del empleado (UNIQUE)
activo	TINYINT	1	Estado del empleado: 1=activo, 0=inactivo
fecha_registro	DATETIME	-	Fecha y hora de registro del empleado
fecha_actualizacion	DATETIME	-	Fecha y hora de última actualización

FUENTE: Elaboración propia, 2025

Tabla 11 : Diccionario de Datos (Historial_transacciones)

CAMPO	TIPO	TAMAÑO	DESCRIPCIÓN
id	INT	11	Identificador único de la transacción (PK)
empleado_id	INT	11	ID del empleado que realizó la transacción (FK)
metros_dispensados	DECIMAL	10,2	Cantidad de metros dispensados
metros_antes	DECIMAL	10,2	Material disponible antes del dispensado
metros_despues	DECIMAL	10,2	Material disponible después del dispensado
fecha	DATE	-	Fecha de la transacción
hora	TIME	-	Hora de la transacción
timestamp	DATETIME	-	Marca de tiempo completa

FUENTE: Elaboración propia, 2025

Tabla 12 :Diccionario de Datos (Configuración)

CAMPO	TIPO	TAMAÑO	DESCRIPCIÓN
id	INT	11	Identificador de configuración (PK)
material_disponible	DECIMAL	10,2	Metros de material actualmente disponibles
capacidad_maxima	DECIMAL	10,2	Capacidad máxima del rollo (30 metros)
alerta_bajo	DECIMAL	10,2	Umbral para alerta de material bajo (8m)
alerta_critico	DECIMAL	10,2	Umbral para alerta crítica (2m)

FUENTE: Elaboración propia, 2025

Tabla 13: Diccionario de Datos (usuarios)

CAMPO	TIPO	TAMAÑO	DESCRIPCIÓN
id	INT	11	Identificador único del usuario administrador (PK)
username	VARCHAR	50	Nombre de usuario para login (UNIQUE)
password	VARCHAR	255	Contraseña encriptada del usuario
nombre	VARCHAR	100	Nombre completo del administrador
rol	VARCHAR	20	Rol del usuario (admin, operador)
activo	TINYINT	1	Estado del usuario: 1=activo, 0=inactivo

FUENTE: Elaboración propia, 2025

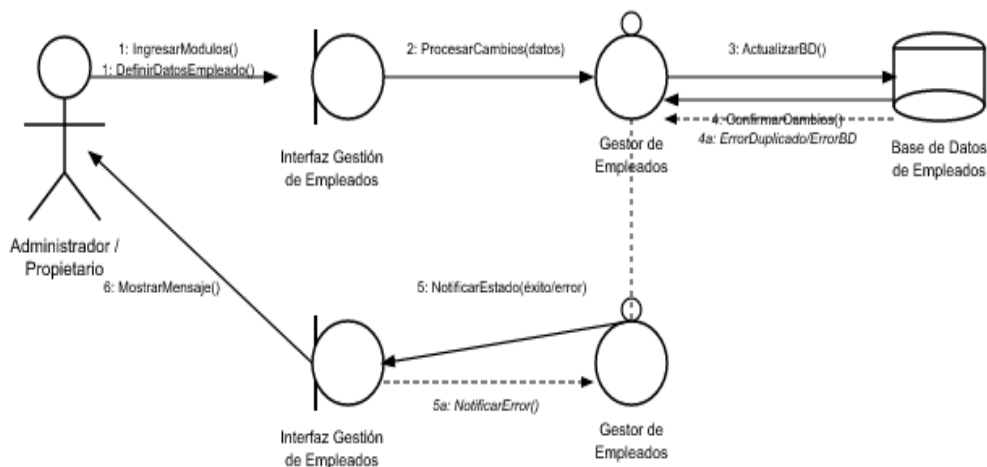
3.2.3 FASE 3: MODELADO DE PROCESOS

La Fase de Modelado de Procesos representa las interacciones dinámicas entre los componentes del sistema mediante diagramas UML. Los diagramas de colaboración muestran los mensajes intercambiados entre objetos para cada proceso, mientras que los diagramas de secuencia ilustran el orden temporal de dichas interacciones.

3.2.3.1 Diagrama de colaboración

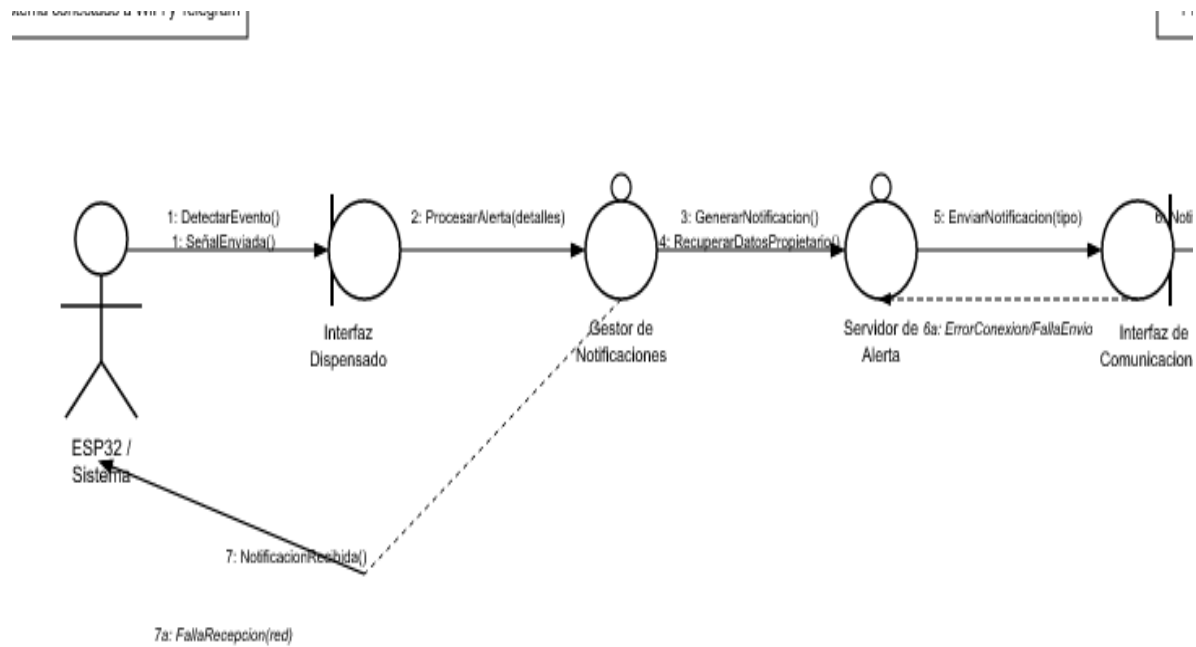
Los diagramas de colaboración describen los mensajes que se intercambian entre los objetos del sistema para completar los tres procesos principales: validación de código de empleado, dispensado de material y consulta de reportes por el administrador.

Figura 25 : Diagrama de Colaboracion – Validación de código de empleado



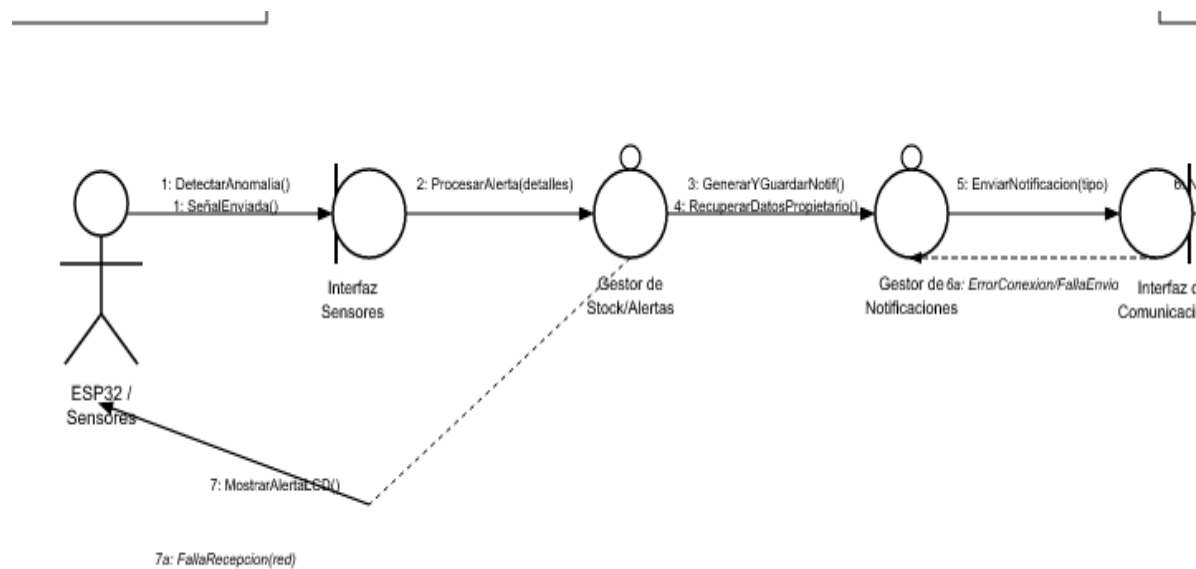
FUENTE: Elaboración propia, 2025

Figura 26 : Diagrama de Colaboración – Dispensado de Material



FUENTE: Elaboración propia, 2025

Figura 27: Diagrama de Colaboracion - consulta de reportes



3.2.4 FASE 4: GENERACION DE LA APLICACION WEB

La Fase 4 del Modelo DRA comprende la construcción e implementación de la interfaz web del sistema. Se desarrolla el dashboard de administración utilizando Node.js con Express.js y el motor de plantillas EJS, permitiendo al administrador gestionar empleados, consultar el historial de transacciones, generar reportes PDF y configurar los parametros del sistema desde cualquier navegador web en la red local.

3.2.4.1 Dashboard de Administración Web

El panel de administración incluye: (1) Autenticación con usuario y contraseña almacenados en SQLite; (2) Gestión de empleados con codigos PIN individuales; (3) Historial de transacciones con filtrado por fecha y empleado; (4) Generación de reportes PDF descargables con estadísticas de consumo; y (5) Configuración de umbrales de alerta de stock y parametros del bot de Telegram.

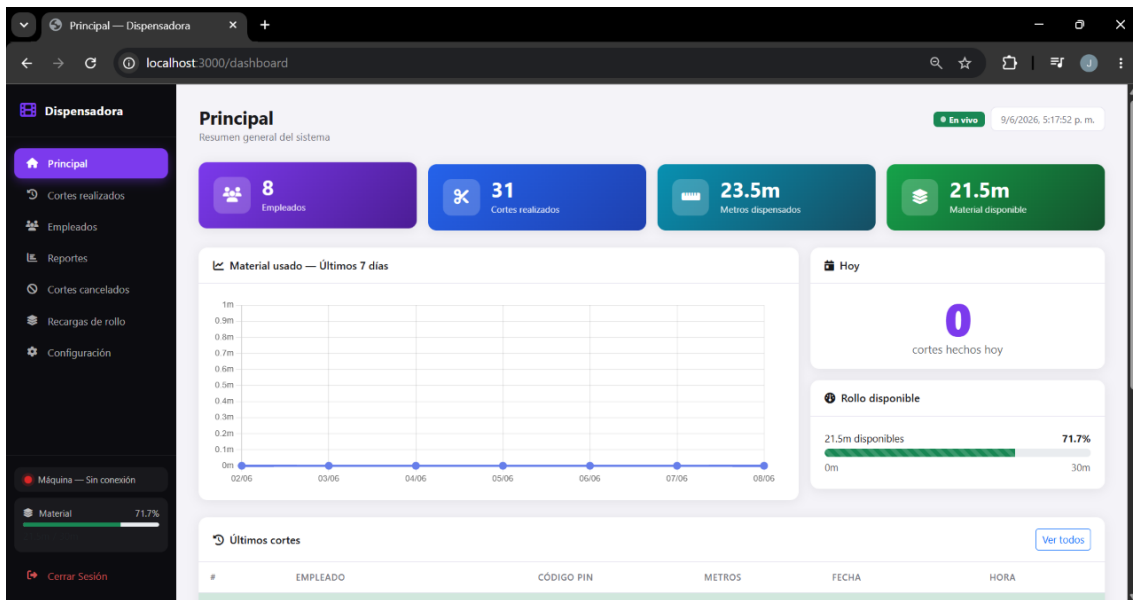


Figura X : Vista Principal: Resumen general del sistema (localhost:3000/dashboard)

Pantalla de inicio del panel administrativo. Muestra tarjetas con metricas en tiempo real: total de empleados, cortes realizados, metros dispensados y material disponible. Incluye

grafico de consumo de los ultimos 7 dias, estado de conexion de la maquina y tabla de ultimos cortes realizados.

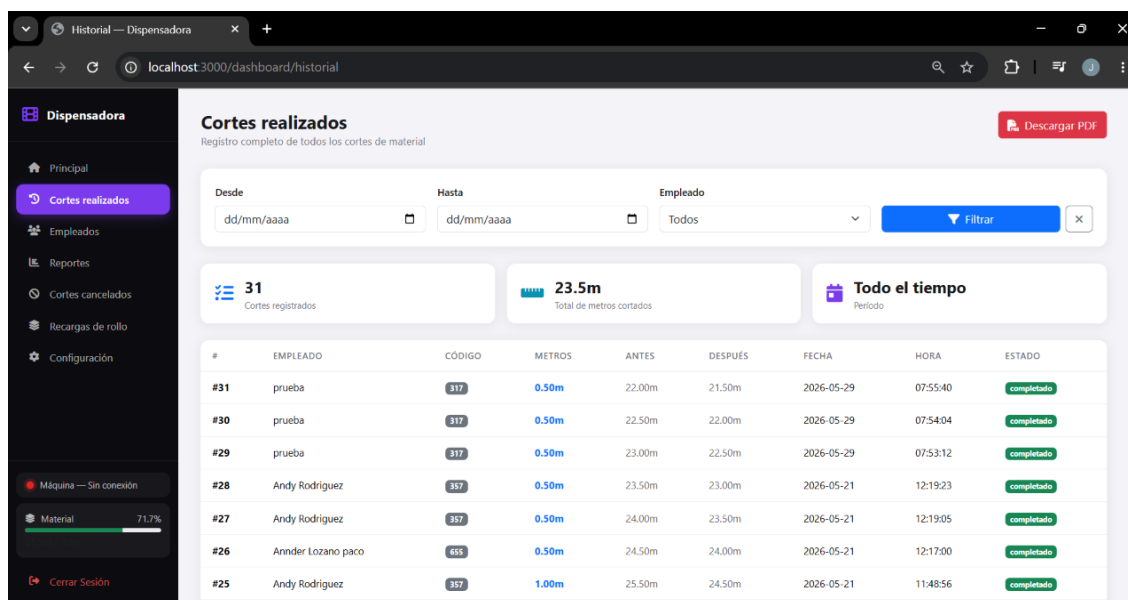


Figura X : Vista Cortes Realizados: Historial completo de dispensados (/dashboard/historial)

Registro completo de todos los cortes de material. Permite filtrar por rango de fechas y empleado. Muestra columnas: empleado, codigo PIN, metros solicitados, metros antes/despues, fecha, hora y estado completado.

Empleados
Gestion de empleados y códigos de acceso

[Nuevo Empleado](#)

Buscar por nombre o código...

ID	NOMBRE	CÓDIGO	ESTADO	REGISTRO	ACCIONES
#6	Andy Rodriguez	357	Activo	2026-05-07	Eliminar Actualizar Editar Ver
#3	Annder Lozano paco	655	Activo	2026-04-16	Eliminar Actualizar Editar Ver
#5	Deyvi daniel	456	Activo	2026-05-07	Eliminar Actualizar Editar Ver
#10	ISACC MAMANI LOZANO	722	Activo	2026-05-25	Eliminar Actualizar Editar Ver
#9	Jhonar lozano	748	Activo	2026-05-21	Eliminar Actualizar Editar Ver
#8	mario	875	Activo	2026-05-14	Eliminar Actualizar Editar Ver
#7	neyer	786	Activo	2026-05-07	Eliminar Actualizar Editar Ver
#11	prueba	317	Activo	2026-05-29	Eliminar Actualizar Editar Ver

Figura X : Vista Empleados: Gestion de empleados (/dashboard/empleados)

Lista todos los empleados con su codigo PIN y estado. Permite buscar por nombre o codigo y realizar acciones: desactivar, copiar link del portal, editar o eliminar.

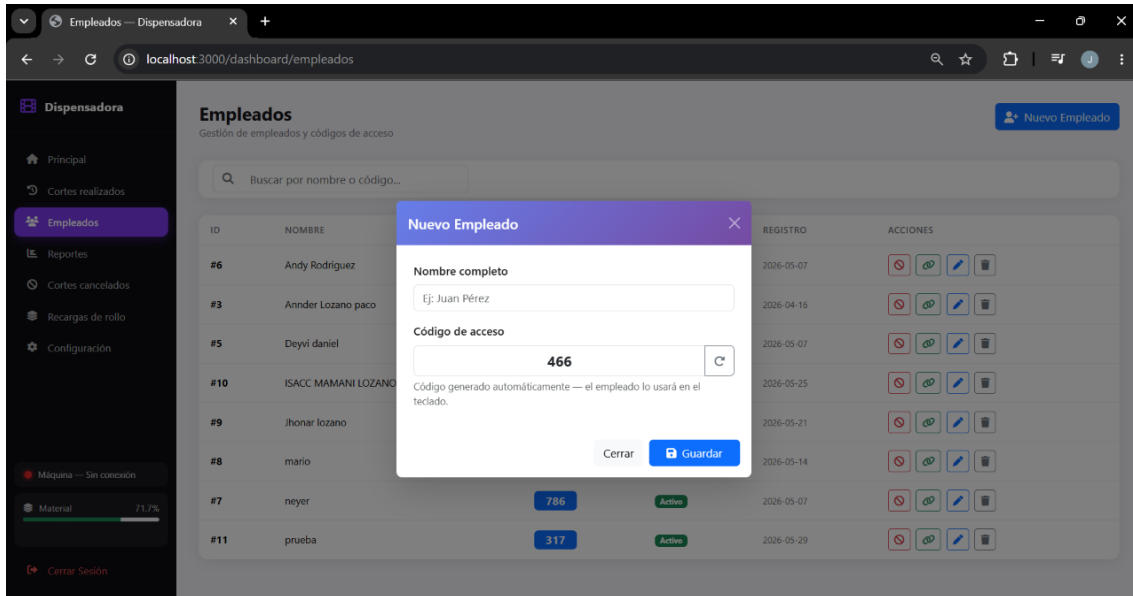


Figura X : Vista Agregar Empleado: Registro de nuevo empleado

Formulario modal para registrar un nuevo empleado. El administrador ingresa el nombre y el sistema genera automaticamente un codigo PIN de 3 digitos que el empleado usara en el teclado matricial del dispensador.

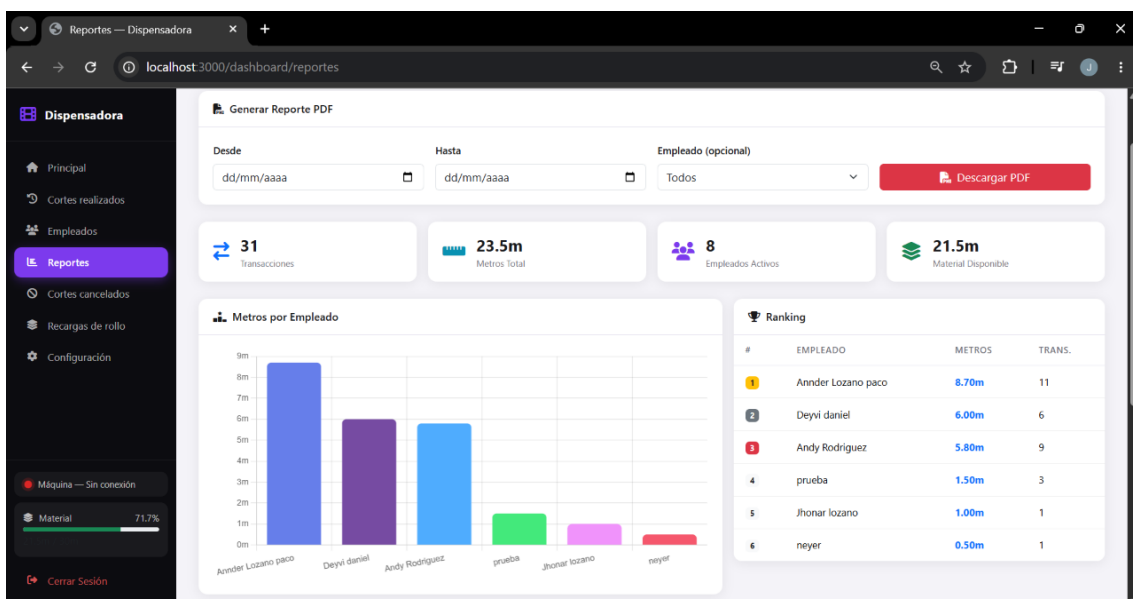


Figura X : Vista Reportes: Estadísticas y generacion de PDF (/dashboard/reportes)

Panel con grafico de barras de metros consumidos por empleado, ranking de consumo y resumen global. Permite filtrar por periodo y descargar un reporte PDF completo.

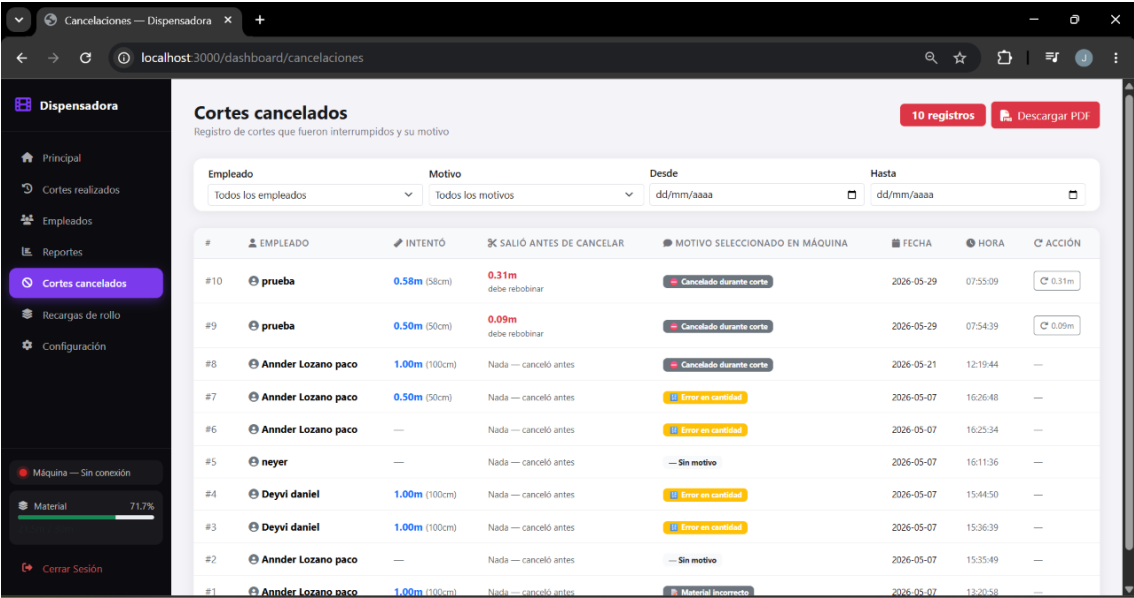


Figura X : Vista Cortes Cancelados: Operaciones interrumpidas (/dashboard/cancelaciones)

Historial de operaciones canceladas. Registra empleado, metros intentados, lo dispensado antes de cancelar y el motivo: cancelado en maquina, error, sin motivo o material incorrecto.

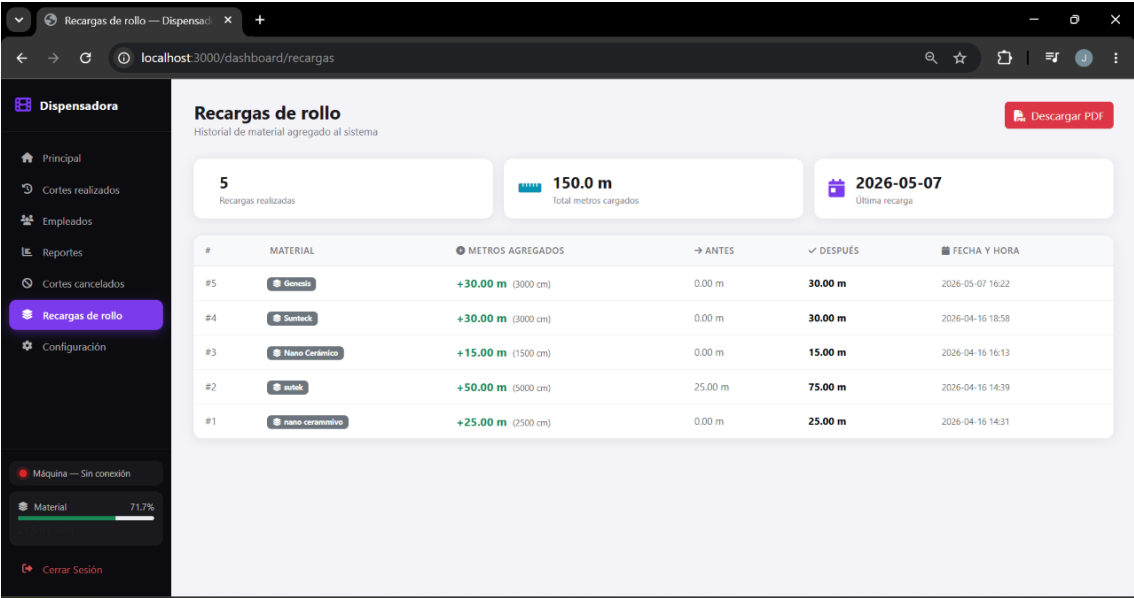


Figura X : Vista Recargas de Rollo: Historial de material cargado (/dashboard/recargas)

Registro de cada recarga de material al sistema. Muestra tipo de material, metros agregados, stock antes y despues, y fecha y hora del evento.

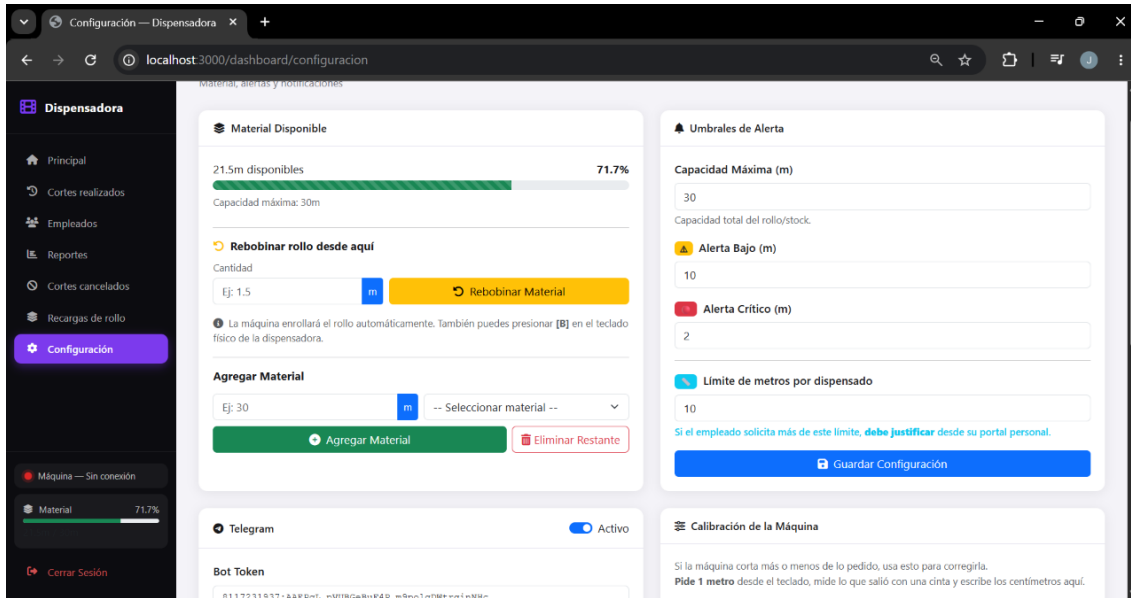


Figura X : Vista Configuración: Parametros del sistema (/dashboard/configuracion)

Panel de configuración: rebobinar rollo, agregar material, definir umbrales de alerta, configurar Bot Telegram, calibrar pasos/metro del motor y descargar backup de la base de datos SQLite.

3.2.4.2 Implementación con Node.js y Express.js

El servidor web se implementa con Node.js v22 utilizando el módulo nativo node:sqlite (DatabaseSync) para el acceso a la base de datos SQLite. Las rutas REST se definen con Express.js: /login, /api/empleados, /api/transacciones, /api/reportes y /api/config. Las vistas se renderizan con EJS generando HTML dinámico. El servidor opera en el puerto 3000 y recibe peticiones HTTP del ESP32 en la red local Wi-Fi.

3.3 ARQUITECTURA DEL SISTEMA

La arquitectura del sistema de dispensadora automatizada integra componentes de hardware y software que operan de forma coordinada para dispensar material de polarizado de manera precisa y registrar cada transacción. La Figura 30 muestra la interacción entre el microcontrolador ESP32, los periféricos de entrada/salida y el servidor Node.js con su base de datos SQLite.

IAMGEN

3.3.1 Componentes de Hardware

Los componentes de hardware del sistema son: (1) Microcontrolador ESP32, que centraliza la lógica de control, gestiona la comunicación Wi-Fi y coordina todos los periféricos; (2) Motor paso a paso 28BYJ-48 con driver ULN2003, que ejecuta el dispensado de material con una precisión de ± 2 cm por metro; (3) Teclado matricial 4x4, que permite al empleado ingresar su código PIN de acceso; (4) Pantalla LCD 20x4 con módulo I2C, que muestra mensajes de bienvenida, estado del sistema y confirmaciones de dispensado; y (5) Sensor de fin de carrera, que detecta el agotamiento del material en el rollo.

3.3.2 Componentes de Software

Los componentes de software del sistema son: (1) Firmware ESP32 (Arduino C++), que implementa la lógica de validación de códigos, control de motores y comunicación HTTP con el servidor; (2) Servidor Node.js con Express.js, que expone la API REST y sirve el panel de administración web; (3) Base de datos SQLite (archivo dispensadora.db), que almacena empleados, transacciones, recargas y configuración del sistema; (4) Motor de plantillas EJS, que genera las vistas HTML del dashboard; y (5) Servicio de Telegram Bot API (via Axios), que envía notificaciones automáticas al administrador ante cada dispensado, recarga o acceso inválido.

3.3.3 Flujo de Comunicación

El flujo de comunicación del sistema sigue el siguiente proceso: el empleado ingresa su código PIN en el teclado del ESP32, el cual envía una petición HTTP POST al endpoint

/api/validar_codigo del servidor Node.js a través de la red Wi-Fi local. El servidor consulta la base de datos SQLite y responde con los datos del empleado o un error de acceso. Si la validación es exitosa, el empleado selecciona los metros a dispensar y el ESP32 envía una segunda petición a /api/registrar_transaccion. El servidor ejecuta la operación de forma atómica (actualiza inventario y registra la transacción en una sola transacción SQLite), responde al ESP32 para activar los motores y envía la notificación de confirmación al administrador vía Telegram Bot API.

La Figura 30 ilustra el diagrama de flujo de comunicación completo del sistema, dividido en tres capas: la capa física (ESP32 y periféricos), la capa de red (Wi-Fi 802.11 b/g/n) y la capa de aplicación (servidor Node.js con base de datos SQLite).

En la capa física, el microcontrolador ESP32 actúa como cliente HTTP. Al recibir un código PIN desde el teclado matricial 4x4, construye una petición HTTP POST con payload JSON y lo transmite al servidor a través de la red Wi-Fi local. La dirección IP del servidor es configurada estáticamente en el firmware del ESP32 para garantizar estabilidad en la comunicación.

En la capa de aplicación, el servidor Node.js recibe la petición en el endpoint /api/validar_codigo, ejecuta la consulta SQL en la base de datos SQLite y devuelve una respuesta JSON. Si el dispensado se completa exitosamente, el servidor envía una notificación asíncrona al Bot de Telegram del propietario mediante la Telegram Bot API, informando el nombre del empleado, los metros dispensados y la hora del evento.

3.3.4 Arquitectura MVC (Modelo-Vista-Controlador)

El sistema dispensadora-node implementa el patrón arquitectónico Modelo-Vista-Controlador (MVC), que organiza el software en tres capas independientes con responsabilidades claramente definidas. Esta separación facilita el mantenimiento, la escalabilidad y la verificación de cada componente de forma aislada, siguiendo las mejores prácticas del desarrollo web moderno con Node.js y Express.js.

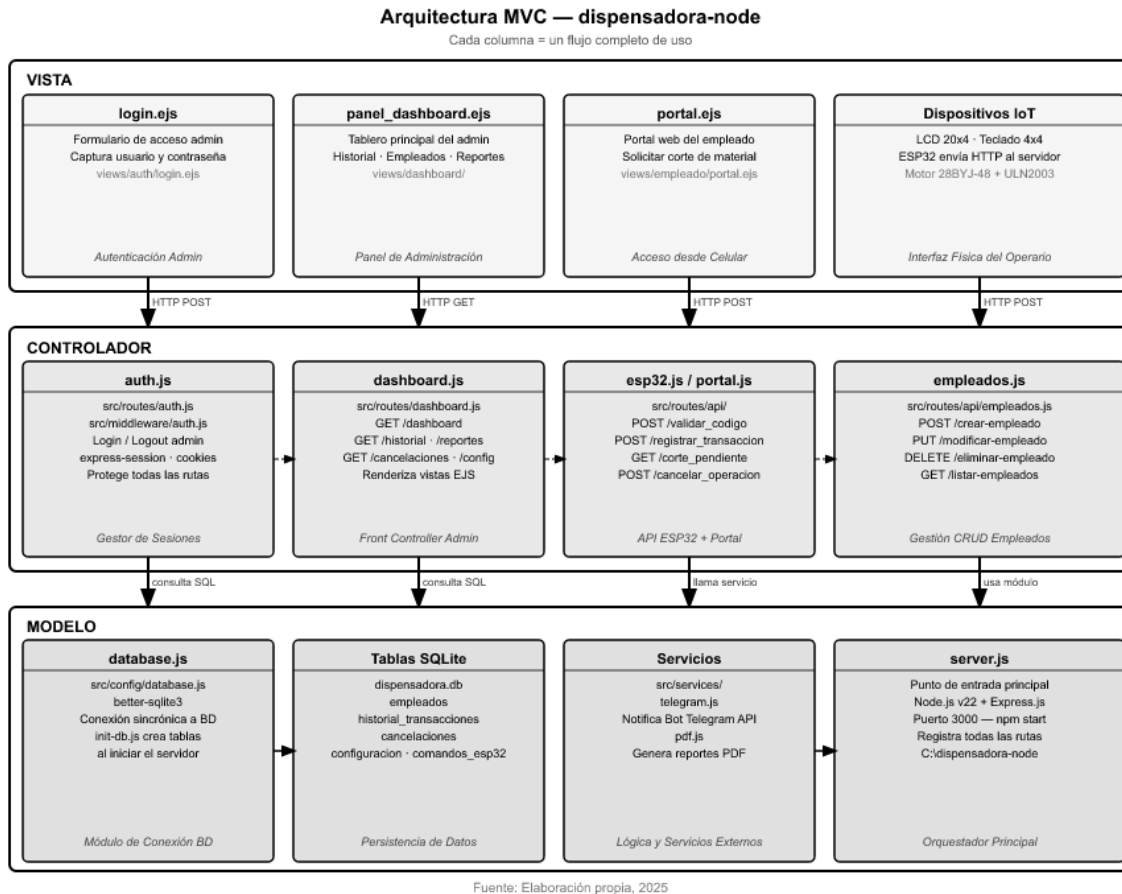


Figura 3.x. Diagrama de Arquitectura MVC — Sistema Dispensadora de Polarizado

Modelo (Model): Representa la capa de datos del sistema. Está implementada con SQLite a través de la librería better-sqlite3, gestionada en el archivo `src/config/database.js`. Las tablas que conforman el modelo son: `empleados` (almacena códigos PIN, nombres, tokens de acceso y cantidades pendientes), `historial_transacciones` (registra cada dispensado con metros, fecha, hora y justificación), `cancelaciones` (guarda las operaciones canceladas con su motivo), `configuración` (controla el stock disponible, capacidad máxima y umbrales de alerta) y `comandos_esp32` (almacena órdenes de rebobinado pendientes de ejecución). El Modelo no tiene conocimiento de la interfaz de usuario ni de las rutas; solo expone los datos al Controlador.

Vista (View): Representa la capa de presentación del sistema. Está implementada con el motor de plantillas EJS (Embedded JavaScript), que genera HTML dinámico en el servidor. Las vistas del sistema son: el dashboard del administrador (`views/dashboard/`), que incluye las

pantallas de historial de transacciones, gestión de empleados, reportes, cancelaciones y configuración del material; y el portal personal del empleado (views/empleado/portal.ejs), accesible desde el celular mediante un enlace único. Además, la pantalla LCD física del ESP32 actúa como Vista del dispositivo de campo, mostrando mensajes como “CORTANDO...”, “COMPLETADO” o “ERROR” directamente al operario.

Controlador (Controller): Representa la capa de lógica del negocio y actúa como puente entre el Modelo y la Vista. Está implementada mediante las rutas Express en src/routes/api/. Los controladores principales son: esp32.js, que expone los endpoints /validar_codigo, /registrar_transaccion, /heartbeat, /corte_pendiente, /stock y /cancelar_operacion para la comunicación con el microcontrolador; portal.js, que gestiona las solicitudes del empleado desde su celular; y empleados.js, que administra el alta, baja y modificación de empleados. Cada controlador recibe la solicitud HTTP, consulta o actualiza el Modelo y devuelve la respuesta a la Vista o al dispositivo solicitante.

La Tabla 1 resume cómo se mapea cada capa MVC a los componentes reales del sistema:

Capa MVC	Tecnología / Archivo	Responsabilidad
Modelo	SQLite + better-sqlite3 (src/config/database.js)	Almacena empleados, transacciones, stock, cancelaciones y comandos
Vista	Plantillas EJS (views/dashboard/ y views/empleado/) + LCD ESP32	Presenta información al administrador, al empleado y al operario físico
Controlador	Rutas Express (src/routes/api/esp32.js, portal.js, empleados.js)	Procesa solicitudes HTTP, aplica lógica de negocio y coordina Modelo y Vista

Flujo de ejemplo — Validación de un empleado: El empleado ingresa su código PIN en el teclado físico del ESP32 y presiona la tecla de confirmación (#). El microcontrolador envía una

solicitud HTTP POST al Controlador (esp32.js, endpoint /validar_codigo). El Controlador consulta al Modelo mediante una sentencia SELECT sobre la tabla empleados. El Modelo devuelve los datos del empleado si existe y está activo. El Controlador construye la respuesta JSON y la devuelve al ESP32. Finalmente, la Vista (pantalla LCD) muestra el mensaje “Hola, [Nombre]” al operario, completando el ciclo MVC.

3.3.5 Descripción del Funcionamiento del Sistema

El sistema de dispensación automatizada de polarizado opera mediante la interacción de tres componentes principales: la unidad física controlada por el microcontrolador ESP32, el servidor de gestión desarrollado en Node.js y las interfaces de usuario accesibles desde dispositivos conectados a la red local.

Proceso de dispensación desde la máquina física. El empleado ingresa su código de identificación mediante el teclado matricial 4×4 instalado en la máquina. El microcontrolador ESP32 envía el código al servidor a través de una petición HTTP POST hacia el endpoint /api/validar_codigo. El servidor consulta la base de datos SQLite, verifica que el código exista y que el empleado esté activo, y devuelve una respuesta de autorización. Una vez validado, la pantalla LCD de 20×4 caracteres muestra el nombre del empleado y el stock disponible. El empleado ingresa la cantidad de metros requerida y confirma con la tecla #. El ESP32 calcula el número de pasos necesarios según la calibración almacenada y activa el motor paso a paso 28BYJ-48 mediante el driver ULN2003, ejecutando la secuencia de medio paso en ocho fases. Al completar el corte, el servidor registra la transacción en la tabla historial_transacciones, descuenta el material del stock y notifica al administrador vía Telegram.

Proceso de dispensación desde el portal móvil. El empleado accede al portal personal desde su teléfono celular mediante una URL única generada con token de identificación. Selecciona la cantidad de metros deseada y presiona el botón de envío. El servidor almacena la cantidad en el campo cantidad_pendiente del empleado en la base de datos. El ESP32 consulta periódicamente el endpoint /api/corte_pendiente cada cuatro segundos; al

detectar una solicitud pendiente, ejecuta el corte de forma automática sin requerir intervención en el teclado físico.

Proceso de cancelación y rebobinado. Si el empleado cancela el corte mientras el motor está en operación, el ESP32 detiene el avance, calcula los metros reales dispensados antes de la cancelación y ejecuta la secuencia inversa del motor para rebobinar exactamente ese tramo de material. El sistema registra en la tabla cancelaciones la cantidad solicitada, los metros efectivamente dispensados y el motivo de cancelación. Tanto el administrador desde el panel de control como el empleado desde su portal pueden visualizar este dato y enviar una orden de rebobinado adicional si fuera necesario.

Panel de administración. El administrador accede al panel mediante autenticación por usuario y contraseña. El sistema le permite visualizar el historial completo de cortes con filtros por empleado y fecha, gestionar el registro de empleados, controlar el stock del rollo, configurar los umbrales de alerta, calibrar la precisión del motor y descargar reportes en formato PDF. El panel muestra en tiempo real el estado de conexión de la máquina mediante señales de latido (heartbeat) enviadas cada diez segundos por el ESP32.

3.4 PRUEBAS DEL SISTEMA

Con el propósito de verificar el correcto funcionamiento del sistema de dispensado automático IoT, se ejecutaron diez casos de prueba funcionales. Las pruebas se realizaron con el ESP32 conectado a la red Wi-Fi local, el servidor Node.js activo y la base de datos SQLite inicializada con datos de prueba.

Tabla: Casos de Prueba del Sistema Dispensadora Automática IoT

N°	ID	Caso de Prueba	Resultado Esperado	Estado
1	CP-001	Autenticación con PIN valido: El sistema valida el	El sistema muestra el nombre del	Aprobado

		código PIN del empleado.	empleado en el LCD y habilita el dispensado.	
2	CP-002	Rechazo de PIN invalido: El sistema rechaza el acceso cuando el PIN no existe.	El sistema muestra acceso denegado en el LCD y envia alerta via Telegram.	Aprobado
3	CP-003	Dispensado de material: El sistema activa el motor y dispensa la cantidad solicitada.	El motor dispensa la cantidad exacta con margen de +/-2 cm.	Aprobado
4	CP-004	Registro de transacción: El sistema registra cada dispensado en la base de datos SQLite.	La tabla transacciones almacena empleado, metros, fecha y hora.	Aprobado
5	CP-005	Notificación Telegram: El sistema envia alerta al administrador tras cada dispensado.	El administrador recibe mensaje con nombre, metros dispensados y hora.	Aprobado

6	CP-006	Actualización de inventario: El sistema descuenta metros del stock disponible.	El dashboard muestra el stock actualizado despues del dispensado.	Aprobado
7	CP-007	Alerta de stock critico: El sistema detecta nivel bajo de material.	El sistema envia alerta via Telegram indicando nivel critico de stock.	Aprobado
8	CP-008	Generación de reporte PDF: El sistema genera reporte del historial de transacciones.	El sistema produce un PDF descargable con transacciones del período.	Aprobado
9	CP-009	Gestión de empleados CRUD: El sistema administra el registro de empleados.	El sistema almacena el nuevo empleado y lo muestra en el panel web.	Aprobado
10	CP-010	Visualizacion en LCD: El sistema actualiza la pantalla en cada etapa del proceso.	La pantalla LCD muestra la bienvenida e instruccion de ingreso de PIN.	Aprobado

3.5 CONCLUSIONES DEL CAPÍTULO

En el presente capítulo se desarrolló la propuesta de solución tecnológica para el problema identificado en talleres de polarizado automotriz. Se presentaron dos modelos de negocio (actual y alternativo), evidenciando las mejoras significativas que aporta la automatización mediante tecnología IoT.

El Modelo DRA permitió estructurar el desarrollo en tres fases bien definidas: Modelado de Gestión, donde se identificaron diez requerimientos funcionales y diez no funcionales; Modelado de Datos, donde se diseñó la estructura de la base de datos con cinco tablas principales; y Modelado de Procesos, donde se representaron las interacciones mediante diagramas UML de colaboración y secuencia.

La arquitectura propuesta, basada en ESP32 como controlador principal, SQLite como sistema de almacenamiento embebido y Node.js/Express como capa de aplicación web, garantiza escalabilidad, confiabilidad y facilidad de mantenimiento sin dependencia de servidores externos. La integración con Telegram Bot API añade valor al sistema al proporcionar notificaciones en tiempo real sobre cada dispensado y alerta de inventario.

Los diagramas de casos de uso, clases, entidad-relación, colaboración y secuencia presentados en este capítulo proporcionan una visión completa del funcionamiento del sistema, facilitando tanto su implementación como su mantenimiento y futuras mejoras.

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

4.1 CONCLUSIONES

El desarrollo del sistema automatizado de control y dispensado de material para polarizado automotriz permitio dar respuesta a una problematica real y concreta que afecta a los talleres del sector: la perdida de material por uso no registrado y la ausencia de trazabilidad en el consumo diario. A partir del trabajo realizado a lo largo de los tres capitulos anteriores, se establecen las siguientes conclusiones.

La elaboracion del modelado de negocio actual y alternativo evidencio con claridad las diferencias entre el proceso manual que se aplica actualmente en los talleres de raybanizado

y el proceso automatizado propuesto. El modelo actual depende completamente de la supervisión directa del propietario y de la confianza hacia el personal, lo que genera condiciones propicias para el desperdicio y el uso no autorizado del material. El modelo alternativo, basado en tecnología IoT, elimina esa dependencia al registrar cada dispensado de forma automática e inmediata, sin posibilidad de manipulación manual del registro.

El módulo de gestión de usuarios con autenticación mediante códigos PIN individuales funciona de manera estable y confiable. Cada empleado accede al sistema únicamente con su código personal, lo que permite identificar con precisión quien retiró material, en qué cantidad y en qué momento. Esta trazabilidad individual convierte al sistema en una herramienta objetiva que protege tanto al propietario como al personal honesto, ya que los datos quedan respaldados en la base de datos SQLite sin posibilidad de alteración posterior.

El módulo de control del motor paso a paso 28BYJ-48 logra una precisión de dispensado de más o menos 2 centímetros por metro, resultado que supera ampliamente la precisión que alcanza un operador realizando el mismo proceso de forma manual, donde el margen de error ronda entre 10 y 20 centímetros. Este nivel de exactitud reduce directamente el desperdicio de material y hace que cada rollo rinda lo que se espera según su capacidad declarada.

El prototipo físico del sistema dispensador demostró ser funcional en condiciones reales de operación. La integración del microcontrolador ESP32 con el driver ULN2003, la pantalla LCD 20x4 y el teclado matricial 4x4 opera de forma coordinada sin interrupciones. El sistema responde a la validación del código PIN en menos de dos segundos y completa el proceso de dispensado con confirmación en pantalla, cumpliendo el requisito de usabilidad planteado desde el inicio del proyecto.

La interfaz web desarrollada con Node.js, Express y el motor de plantillas EJS entrega al administrador una herramienta práctica para consultar el historial de transacciones, gestionar los empleados registrados y generar reportes de consumo en formato PDF. El dashboard responde en tiempo real a cada dispensado realizado desde el ESP32, lo que

permite al propietario conocer el estado del inventario desde cualquier dispositivo conectado a la red local, sin necesidad de estar físicamente presente en el taller.

Las pruebas funcionales ejecutadas sobre el sistema arrojaron resultados satisfactorios en los diez casos de prueba planteados. Se verificó la autenticación correcta con PIN válido, el rechazo de códigos inexistentes con alerta vía Telegram, el registro exacto de cada transacción en la base de datos, la actualización del inventario en el dashboard, la detección de stock crítico y la generación del reporte PDF. El sistema mantuvo estabilidad en la comunicación WiFi entre el ESP32 y el servidor Node.js durante todas las pruebas realizadas.

En términos generales, el sistema desarrollado cumple con el objetivo general planteado: mejorar la administración del inventario en talleres de raybanizado mediante la automatización del dispensado y el registro digital de cada transacción. La solución resulta económicamente accesible, técnicamente viable y operativamente simple, lo que la convierte en una alternativa real y replicable para el sector.

4.2 RECOMENDACIONES

A partir de los resultados obtenidos y la experiencia acumulada durante el desarrollo del proyecto, se identifican las siguientes recomendaciones orientadas a mejorar, escalar y mantener el sistema a futuro.

Se recomienda incorporar un sensor de fin de carrera o de peso que detecte cuando el rollo de material está por agotarse completamente, de manera que el sistema genere una alerta automática antes de que el stock llegue a cero. Esto evitaría interrupciones en el servicio al cliente por falta de material y permitiría al propietario planificar la recarga con anticipación.

Resultaría conveniente ampliar el sistema para soportar múltiples tipos de material en paralelo, ya que los talleres de raybanizado trabajan con distintas variedades de láminas según el tipo de vehículo y el nivel de oscurecimiento requerido. Una versión futura del sistema podría gestionar varios rollos simultáneamente, cada uno con su propio control de stock e historial de dispensado.

Se sugiere implementar actualizaciones de firmware del ESP32 mediante OTA (Over-The-Air), lo que permitiría corregir errores o agregar funcionalidades al microcontrolador de forma remota sin necesidad de conectar físicamente un cable al dispositivo. Esto facilitaría el mantenimiento del sistema una vez instalado en el taller.

Para entornos donde la conexión WiFi no es estable, vale la pena considerar el almacenamiento local de transacciones en la memoria del ESP32 con sincronización posterior al servidor cuando la conexión se restablezca. De esta forma el sistema seguiría registrando dispensados incluso ante cortes momentáneos de la red, sin perder información.

Finalmente, se recomienda que los propietarios de talleres que adopten este sistema realicen una capacitación inicial breve con los empleados antes de la puesta en marcha, no para explicar el funcionamiento técnico sino para comunicar el propósito del sistema: garantizar transparencia y equidad en el uso del material. La experiencia del proyecto mostro que cuando el personal comprende que el sistema los protege a ellos también, la adopción tecnológica resulta mucho más fluida y sin resistencia.

BIBLIOGRAFÍA

Bibliografía

- Anderson, T. (2022). *Material Management Practices in Automotive Window*. Los Angeles: TintPro Publications.
- Arduino. (2024). *Acerca del Software y el IDE*. Obtenido de <https://www.arduino.cc/en/software>
- Association, N. E. (2016). *NEMA Standards Publication MG 1: Motors and Generators*. Arlington, VA: National Electrical Manufacturers Association.
- Barrett, R. (2013). *La organización basada en valores: Liberando el potencial humano para el rendimiento y la rentabilidad*. Nueva York / Londres: Taylor & Francis.
- Chen, P. P. (1979). *The Entity-Relationship Model: Toward a Unified View of Data*. New York: ACM Press (Association for Computing Machinery).
- Cockburn, A. (2020). Bostón: Addison-Wesley Profesional.
- Corporation, O. (2024). *Documentación de SQLite (o Descripción general de SQLite)*. Obtenido de SQLite Documentation: <https://www.sqlite.org/docs.html>
- corporativo (Campo de texto) Red de desarrolladores de Mozilla. (2023). Obtenido de Documentos web de MDN: <https://developer.mozilla.org/es/docs/web/html>.
- Crockford, D. (2006). *JSON.org*. Obtenido de JSON: La alternativa ligera a XML: <https://www.json.org/json.html>
- Developers, G. (2024). *SQLite Documentation*.
- DB Browser for SQLite. Documentación. (2024). Obtenido de DB Browser for SQLite: <https://sqlitebrowser.org/>
- Fielding, R. T. (2000). *Estilos arquitectónicos y el diseño de arquitecturas de software basadas en redes*. Universidad de California, Irvine (UCI).

- Fielding, R. T., & Káiser, G. (1999). *Estilos arquitectónicos y el diseño de arquitecturas de software basadas en redes*. Obtenido de Estilos arquitectónicos y el diseño de arquitecturas de software basadas en redes: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fielding, R. T., Kaiser, G., & al., e. (1999). *Estilos arquitectónicos y el diseño de arquitecturas de software basadas en redes*. Obtenido de Universidad de California, Irvine (UCI): <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fundación de Software Apache*. (2023). Obtenido de La Fundación de Software Apache:: <https://www.apache.org/>
- Gard., S. (2023). *Professional Automotive Window Film Catalog*. California.
- Group, Object Management. (2017). *Unified Modeling Language (UML) Specification*. Obtenido de <https://www.omg.org/spec/UML>
- Hernández Sampieri, R. (2014). *Metodología de la investigación (6ª ed.)*. México D.F.: McGraw-Hill Education.
- Hernández Sampieri, R. (2014). *Metodología de la investigación (6ª ed.)*. Mexico: McGraw-Hill Education.
- Hernández Sampieri, R. (2019). Los Angeles: McGraw-Hill Education.
- J, G., R, B., Marusic, S., & Palaniswami, M. (2013). Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 1645–1660.
- Joyanes Aguilar, L. (2013). *Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos*. Madrid: McGraw-Hill Interamericana.
- Labcenter Electronics*. (2025). Obtenido de Proteus Design Suite: Features (o Documentación Proteus): <https://www.labcenter.com/>

Magazine, W. F. (2022). *Material Management Practices in Automotive Window*. Los Angeles: TintPro Publications.

Magazine., W. F. (2023). *Top 10 Innovative Tinting Companies in the U.S.* <https://www.windowfilmmag.com/>.

Microsoft, C. (2023). *Informe Anual* . Redmond, Washington, EE.UU.

Microsoft, C. (2023). *Informe Anual 2023*. Estados_Unidos: Redmond, Washington, EE.UU.

MicroSystems, A. (2020). *Folleto de Oferta Pública Inicial de Acciones Ordinarias*. Wilmington, Massachusetts: Allegro MicroSystems, Inc.

(2016). Monje, EP Jr. (R. A. Sociología, Entrevistador)

Object Management Group. (2017). *Object Management Group*. Obtenido de UML 2.5.1 Superstructure Specification: <https://www.omg.org/spec/UML>

Ogata, K. (2010). *Ingeniería de control moderna*. México D.F.: Pearson Educación.

Express.js Framework. (2023). Obtenido de Documentación de PHP (o Manual de PHP): <https://www.php.net/docs.php>

PlatformIO, L. (2023). *Su puerta de entrada a la excelencia en el desarrollo de software integrado*. Obtenido de PlatformIO: <https://docs.platformio.org/en/latest/>

Postmasn INC. (2023). Obtenido de Cartero, Inc.: <https://www.postman.com/product/>

Pressman, R. S. (2010). *Ingeniería del Software: Un Enfoque Práctico*. McGraw-Hill.

Pressman, R. S. (2014). *Ingeniería del software*. Un enfoque práctico (7ª ed.). McGraw-Hill.

Pressman, R. S. (2014). *Ingeniería del software*. Un enfoque práctico (7ª ed.). McGraw-Hill.

Royce, W. W. (1970). Gestión del desarrollo de grandes sistemas de software. *conferencia/revista Proceedings of IEEE WESCON (o la revista donde fue reimpresso, como IEEE Transactions on Software Engineering)*.

Systems, E. (2021). *ESP32 Technical Reference Manual*. Espressif Systems.

Systems, E. (2023). *ESP32 Series Datasheet*. Shanghai: Espressif Systems.

Systems., E. (2021). *ESP32 Technical Reference Manual*. Los Angeles: Espressif Systems.

Telegrama. (2023). Obtenido de Telegram Messenger LLP: <https://telegram.org/faq>

Ulrich, K. T., & Eppinger, S. D. (2015). *Product Design and Development (6th ed.)*. New York, NY: McGraw-Hill Education.

Node.js: Documentación. (2024). Obtenido de OpenJS Foundation - Node.js: <https://nodejs.org/en/docs>

Requerimientos No Funcionales

- **Disponibilidad:** El sistema opera el 95% del tiempo durante horario laboral.
- **Tiempo de respuesta:** Validación de PIN en menos de 2 segundos.
- **Precisión:** Dispensado con error máximo de $\pm 2\text{cm}$.
- **Confiabilidad:** Registro exitoso del 99% de las transacciones.
- **Escalabilidad:** Capacidad de agregar hasta 20 usuarios sin degradación de rendimiento.
- **Seguridad:** Encriptación de comunicaciones WiFi y protección de base de datos.
- **Usabilidad:** Interfaz intuitiva con tiempo de capacitación menor a 15 minutos.

Requerimientos Funcionales

1. El sistema permite autenticación de usuarios mediante código PIN de 3 dígitos.

2. El sistema dispensa cantidades exactas de material según especificación del usuario (10-300 cm).
3. El sistema dispensa la cantidad exacta de material solicitada mediante el control preciso del motor paso a paso.
4. El sistema registra cada transacción con: usuario, cantidad, fecha, hora y material restante.
5. El sistema envía notificación instantánea vía Telegram al propietario por cada uso.
6. El sistema permite consulta de historial de transacciones vía interfaz web.
7. El sistema genera alertas cuando el material disponible sea inferior a 1000cm (10 metros).
8. El sistema permite gestión de usuarios (alta, baja, modificación) desde interfaz web.
9. El sistema funciona sin WiFi guardando datos localmente y sincronizando posteriormente.
10. El sistema incluye botón de emergencia para detención inmediata de motores.

Conclusión

El presente proyecto demuestra que es posible implementar un sistema automatizado de dispensado y control de material en talleres de raybanizado mediante tecnología IoT de bajo costo. El sistema desarrollado integra satisfactoriamente el microcontrolador ESP32, el motor paso a paso 28BYJ-48 con driver ULN2003, una interfaz web Node.js y notificaciones en tiempo real vía Telegram, logrando una precisión de dispensado de ± 2 cm. Las pruebas funcionales ejecutadas sobre el prototipo arrojaron resultados satisfactorios en los diez casos de prueba planteados, verificando la autenticación por código PIN, el registro automático de transacciones y la generación de reportes PDF. Desde la perspectiva económica, el costo de implementación es accesible para pequeñas y medianas empresas del sector, con un retorno de inversión estimado de 2 a 4 meses considerando la reducción de pérdidas de material. En conclusión, el sistema automatizado representa una solución tecnológica viable, replicable y de impacto directo en la rentabilidad operativa de talleres de polarizado automotriz en Bolivia.